

Luna_HiFi Tricycle

A line-by-line manual for the repository, Isaac Lab, Newton MPM
soil and rsl_rl PPO

Generated for the hvnsel/newtonRL repository

3 September 2026

matches branch `claude/luna-hifi-isaac-runbook-imdm7y`

Contents

How to read this manual	4
I The big picture	6
1 The repository	7
1.1 What is in it	7
1.2 When does each file run?	8
1.3 Tricycle and excavation are independent	8
1.4 The runbook and this manual	9
2 Reinforcement learning in ten minutes	10
2.1 The loop	10
2.2 Vectorised environments	10
2.3 Policy and value networks	10
2.4 Discounting and GAE	11
2.5 PPO in plain words	11
2.6 Terminated versus timed out	11
2.7 How the numbers map onto the tricycle	12
3 From isaacsim train to a checkpoint	13
3.1 The command	13
3.2 The whole path at a glance	14
3.3 Step 1: the CLI finds our package	14
3.4 Step 2: configs are built and overridden	15
3.5 Step 3: the environment is built	15
3.6 Step 4: the wrapper	16
3.7 Step 5: the runner	16
3.8 One environment step, in detail	17
3.9 Play	18
4 The physics stack	19
4.1 Layers	19
4.2 The rigid-body side: MuJoCo-Warp	20
4.3 The soil side: implicit MPM	20
4.4 Coupling the two solvers	21
4.5 The collision pipeline and <code>soft_contact_max=0</code>	22
4.6 Time stepping and CUDA graphs	23
4.7 Why only a handful of envs fit	23
5 From MJCF to USD: the car asset	24
5.1 Why two descriptions	24
5.2 Anatomy of the MJCF	24
5.3 The converter	25

5.4	How MJCF names become prim paths	25
5.5	What the new body looks like	26
5.6	Regenerating the USD	27
II	Every file, every line	28
6	pyproject.toml	29
7	luna_hifi_tasks/__init__.py	31
8	tricycle/__init__.py: registering the task	32
9	tricycle.py: the car model	34
9.1	Header	34
9.2	Imports and dimensions	35
9.3	Building the frame	36
9.4	The MJCF string	37
9.5	Writing the file	39
10	tricycle_env_cfg.py: the environment configuration	41
10.1	Header	41
10.2	Imports	42
10.3	Constants	43
10.4	The car	44
10.5	The soil	46
10.6	The hidden slab	47
10.7	The scene	48
10.8	The environment config	49
10.9	__post_init__: the physics	51
10.10	play_mode: settings for playback	54
11	tricycle_env.py: the environment	55
11.1	Header, imports, constructor	55
11.2	Scene hook	56
11.3	Actions	57
11.4	Observations	58
11.5	Reward	59
11.6	Terminations	59
11.7	Reset	60
12	agents/: the PPO configuration	62
12.1	agents/__init__.py	62
12.2	agents/rsl_rl_ppo_cfg.py	62
13	The USD asset files	65
13.1	tricycle/tricycle.usda: the entry point	65
13.2	payloads/base.usda: geometry	66
13.3	payloads/robot.usda: robot schema tags	71
13.4	payloads/Physics/physics.usda: bodies, joints, masses	73
13.5	payloads/Physics/mujoco.usda: MuJoCo variant	77
13.6	payloads/Physics/physx.usda: PhysX variant	80
13.7	config.yaml and .asset_hash	81

14 The runbook commands, flag by flag	83
14.1 One-time setup	83
14.2 Every session	84
14.3 Troubleshooting table, mapped to the code	85
III Appendices	86
A Glossary	87
B What changed in this revision	89
C Tensor shapes cheat sheet	91
D Where to look in the Isaac Lab source	92
E Numbers at a glance	94

How to read this manual

This document explains the `Luna_HiFi` repository as it is used for the tricycle task: how the code is organised, what every line of every tricycle-related file does, and how each piece plugs into Isaac Lab, the Newton physics engine and the `rs1_rl` learning library. It is written for someone who can read Python but has not worked inside Isaac Lab before. Plain language is used on purpose. Where a word has a precise meaning in Isaac Lab it is introduced in a box the first time it appears and it is also listed in the glossary (Appendix A).

Two parts

Part I, the big picture, explains the system from the outside in: the repository layout, what reinforcement learning is, what happens between typing `isaacsim train` and a checkpoint appearing on disk, how the physics is put together, and how the car model becomes a USD asset.

Part II, every file, every line, walks through each file that takes part in tricycle training. Each file is shown in slices with its *real line numbers* (the listings are read straight from the repository at build time, so what you see is exactly what is committed), and every slice is followed by a numbered explanation. Lines that belong together (a closing bracket, a blank line, a run of imports) are explained as a group, but no line is skipped.

Conventions

- `monospace` marks names that appear literally in code, on the command line or in a file path.
- “Line 42” always refers to the line number printed in the listing immediately above it, which is the line number in the actual file.
- Coloured boxes carry background knowledge:

Isaac Lab: a concept from Isaac Lab

Blue boxes explain something Isaac Lab does for you, usually with the file in the Isaac Lab source tree where it is implemented, so you can go and read the original.

Newton: a concept from the physics engine

Purple boxes explain the Newton physics engine, its MuJoCo-Warp rigid body solver, or the implicit Material Point Method (MPM) soil solver.

Why: a design decision

Green boxes answer “why is it written like this?”. Most of them record a lesson that cost real debugging time; they are the same lessons as in `RUNBOOK.md`, section 3, but placed next to the line of code they explain.

Watch out: a trap

Orange boxes flag things that break silently or produce confusing errors.

What is covered and what is not

Everything under `luna_hifi_tasks/tricycle/`, the package root files, `pyproject.toml`, the converted asset under `assets/tricycle/` and the commands in `RUNBOOK.md` are covered line by line. The `luna_hifi_tasks/excavation/` folder is *not* covered. It is a template for the future rover task, it still contains `STUB` and `VERIFY` markers, and nothing in it is imported when the tricycle trains (Chapter 1 shows why).

Versions this manual matches

- Isaac Lab, `develop` branch, early September 2026 (the branch that ships the `isaacsim train` command, the Newton backend and `isaacsim.contrib.coupling`). File paths quoted from Isaac Lab are relative to the Isaac Lab checkout.
- Newton 1.5 (the physics engine Isaac Lab `develop` installs).
- `rs1_rl` 5.x as pulled in by `isaacsim -i`; the configuration in this repository uses the older “policy” style and Isaac Lab converts it on start-up (Chapter 12).
- Python 3.12 on Windows 11, exactly as in `RUNBOOK.md`.

Quoted line numbers of *Isaac Lab* files may drift as the `develop` branch moves; the function and class names are stable enough to search for.

Part I

The big picture

Chapter 1

The repository

1.1 What is in it

The repository is one Python package plus the files needed to install it, one converted robot asset, and documentation. After the clean-up made in this revision the tree looks like this:

Luna_HiFi/	<- git root, pyproject.toml lives here
pyproject.toml	how pip installs the package + the isaacsimlab.
↳ tasks entry point	
.gitignore	keeps __pycache__, *.egg-info and LaTeX build
↳ files out of git	
README_RL.md	the original template notes (superseded, kept
↳ for the excavation map)	
RUNBOOK.md	the validated setup and daily commands
luna_hifi_tasks/	THE PYTHON PACKAGE
__init__.py	imports .tricycle -> registers the task
tricycle/	the pipeline-validation task (this manual)
__init__.py	gym.register("Luna-Tricycle-Direct", ...)
tricycle.py	the car as an MJCF string (source of the USD
↳ asset)	
tricycle_env_cfg.py	scene, physics, spaces, termination knobs
tricycle_env.py	the environment class: actions, obs, reward,
↳ dones, reset	
agents/	
__init__.py	empty; makes "agents" importable
rsl_rl_ppo_cfg.py	PPO hyper-parameters for rsl_rl
excavation/	the future rover task (template, not covered
↳ here)	
assets/	
config.yaml	converter settings that produced the USD (
↳ generated)	
.asset_hash	converter cache key (generated)
tricycle/	
tricycle.usda	the file the env cfg points at
payloads/base.usda	geometry (visible shapes) of the car
payloads/robot.usda	Isaac robot/link/joint schema tags
payloads/Physics/physics.usda	rigid bodies, joints, masses, Newton collision
↳ tags	
payloads/Physics/mujoco.usda	MuJoCo-specific overrides (joint damping)
payloads/Physics/physx.usda	PhysX-specific overrides (unused with Newton)
docs/tricycle_manual/	this document (LaTeX source + PDF)

Three kinds of file live here and it helps to keep them apart:

- **Code you write.** Everything in `luna_hifi_tasks/` and `pyproject.toml`.
- **Generated files you keep.** Everything in `assets/`. They are produced by Isaac Lab's

MJCF-to-USD converter from `tricycle.py` and are committed because the converter needs the Isaac Sim pip package, which not every machine has. When `tricycle.py` changes, these must be regenerated (Chapter 14).

- **Generated files you do not keep.** `__pycache__`/ folders and `luna_hifi_tasks.egg-info/`. They are rebuilt by Python and pip every time and are now ignored by git.

1.2 When does each file run?

A useful way to understand the package is to ask *when* each file is executed. There are four moments.

Install time `pip install -e .` reads `pyproject.toml`. It records two things in the Python environment: that the folder `luna_hifi_tasks` is importable, and that the string "luna_hifi_tasks" is registered under the entry-point group `isaacsim.tasks`. Nothing in the package is executed yet.

Start-up time When you run `isaacsim train ...`, Isaac Lab's command-line tool asks Python for every entry point in the `isaacsim.tasks` group and imports each one. Importing `luna_hifi_tasks` runs `luna_hifi_tasks/__init__.py`, which imports `tricycle/__init__.py`, which calls `gym.register`. From now on Gymnasium knows the name `Luna-Tricycle-Direct` and which classes it maps to. `tricycle/__init__.py` also imports `agents`, so `agents/__init__.py` runs (it is empty).

Configuration time Isaac Lab looks up the registered task, imports `tricycle_env_cfg.py` and `agents/rsl_rl_ppo_cfg.py`, creates one `TricycleEnvCfg` and one `TricyclePPORunnerCfg`, and applies command-line overrides such as `-num_envs 4` to them. Importing `tricycle_env_cfg.py` imports `tricycle.py` for the constants it needs (`CHASSIS_Z`, joint names).

Run time Isaac Lab imports `tricycle_env.py`, builds a `TricycleEnv` from the config, and hands it to `rsl_rl`. From then on the only code of yours that runs is the handful of methods in `TricycleEnv`, called thousands of times per second.

Chapter 3 follows this sequence in detail.

1.3 Tricycle and excavation are independent

The two task folders share nothing at import time. The evidence:

- `luna_hifi_tasks/__init__.py` contains a single line: `from . import tricycle`. The `excavation` package is never imported, so none of its code runs and none of its `VERIFY` markers can affect a tricycle run.
- No file in `tricycle/` mentions `excavation`, and no file in `excavation/` mentions `tricycle`. Each has its own `__init__.py`, its own env, its own cfg and its own `agents/`.
- Until this revision, `tricycle/soil.py` was a byte-for-byte copy of `excavation/soil.py` that nothing in `tricycle/` imported. The tricycle copy has been removed; the excavation copy is untouched (Appendix B).

The only thing they share is the Isaac Lab machinery they both sit on. When the excavation task is finished, adding `from . import excavation` to `luna_hifi_tasks/__init__.py` will register it next to the tricycle.

1.4 The runbook and this manual

`RUNBOOK.md` is the operational document: what to type, in which window, and what the error messages mean. This manual is the explanatory document: why the code is the way it is. Chapter 14 goes through the runbook's commands flag by flag so the two stay connected.

Chapter 2

Reinforcement learning in ten minutes

This chapter gives just enough reinforcement learning (RL) to make every number in the tricycle configuration meaningful. If you know PPO already, skim Section 2.7 for how the general ideas map onto this task.

2.1 The loop

An **environment** is a simulation that has a state, accepts an **action**, advances one **step**, and returns three things: an **observation** (what the agent is allowed to see), a **reward** (one number saying how good that step was) and a **done** flag (whether the episode is over). An **agent** is a function, the **policy**, that maps observations to actions. Training means adjusting the policy so that the sum of rewards over an **episode** (a sequence of steps that ends when done is true) becomes as large as possible.

For the tricycle: the state is the pose and velocity of the car, the joint angles and speeds, and the position of every soil particle. The observation is a 15-number vector (Chapter 11). The action is two numbers in $[-1, 1]$: throttle and steering. The reward is how far forward the car moved during this step, in metres. An episode lasts 2 seconds or until the car flips, drives off the strip, or the physics blows up.

2.2 Vectorised environments

One simulation step of one car teaches the agent very little. Isaac Lab therefore simulates N identical copies of the scene at once, called **environments** or **envs**, laid out on a grid in the same physics world, and returns observations as a tensor of shape $(N, 15)$, rewards as $(N,)$, and so on. Every method you write in `TricycleEnv` operates on whole tensors; there is never a Python loop over envs. Each env has its own **origin** (`scene.env_origins`), and positions are usually measured relative to it so that env 7 does not look different from env 0 just because it sits further along the grid.

The number of envs is a memory decision. On the 6 GB laptop card the soil solver limits it to a handful (Chapter 4); on a large GPU the same code runs thousands.

2.3 Policy and value networks

The policy is a small neural network. In this task it has two hidden layers of 64 units with the ELU activation. Given the 15 observations it outputs the *mean* of a Gaussian distribution over the two actions; a second learned parameter, the standard deviation (initially 1.0, see `init_noise_std`), controls how much the agent explores around that mean. During training actions are *sampled* from this Gaussian, which is where exploration comes from. During play the mean is used.

A second network of the same shape, the **critic** or **value function**, predicts the expected sum of future rewards from a given observation. It is not used to act; it is used to judge whether an action turned out better or worse than expected. That difference is the **advantage**. Both networks are trained together, which is why the architecture is called *actor-critic*.

2.4 Discounting and GAE

Rewards far in the future are worth less than rewards now. The **discount factor** γ (`gamma = 0.99`) multiplies each future reward by γ^k after k steps. With $\gamma = 0.99$ the effective horizon is about $1/(1 - \gamma) = 100$ steps, which at 50 Hz is exactly one 2-second episode.

The advantage estimate mixes the critic’s predictions with actual rewards. **Generalised Advantage Estimation** (GAE) blends one-step and many-step estimates with a second parameter λ (`lam = 0.95`): higher λ trusts observed rewards more (less bias, more noise), lower trusts the critic more.

2.5 PPO in plain words

Proximal Policy Optimisation (PPO) is the learning rule used here. Each **iteration**:

1. **Roll out.** Run the current policy for `num_steps_per_env` steps in every env, storing observations, actions, rewards, dones and the critic’s values. With 4 envs and 50 steps that is 200 samples per iteration.
2. **Compute advantages** with GAE over the stored rollout.
3. **Update.** Split the rollout into `num_mini_batches` mini-batches and pass over them `num_learning_epochs` times. For each mini-batch, compute the ratio between the probability the *new* policy assigns to the stored action and the probability the *old* policy assigned. Multiply by the advantage, but *clip* the ratio to $[1 - \epsilon, 1 + \epsilon]$ with $\epsilon = \text{clip_param} = 0.2$. The clipping is what makes PPO “proximal”: the policy is not allowed to move far from the one that collected the data, which keeps learning stable.
4. Add the critic’s loss (mean squared error to the returns, weighted by `value_loss_coef`) and subtract an **entropy bonus** (`entropy_coef`) that rewards keeping the action distribution wide, i.e. keeps exploring.
5. Take a gradient step with Adam at `learning_rate`, after clipping the gradient norm to `max_grad_norm`.

With `schedule = "adaptive"`, `rsl_rl` also measures the KL divergence between the old and new policies after each mini-batch. If it exceeds $2 \times \text{desired_kl}$ the learning rate is divided by 1.5; if it is below half of `desired_kl` the rate is multiplied by 1.5 (bounded to $[10^{-5}, 10^{-2}]$). This automatic tuning is why the initial `learning_rate = 1e-3` does not need to be exactly right.

2.6 Terminated versus timed out

An episode can end for two very different reasons and RL algorithms must know which one happened:

- **Terminated:** the task itself ended (the car flipped over). The value of the final state is genuinely zero; nothing good can follow.
- **Timed out** (also called truncated): the clock ran out while the car was still driving happily. Its state still has value; the episode was merely cut for bookkeeping.

Isaac Lab’s `_get_dones` therefore returns *two* boolean tensors, `terminated` and `time_out`, and the `rsl_rl` wrapper passes the time-outs to the learner separately so that it can *bootstrap*: add the

critic’s estimate of the remaining value back into the return for timed-out episodes. Getting this wrong makes the agent believe that surviving to 2 seconds is bad, which is the opposite of what we want.

2.7 How the numbers map onto the tricycle

Quantity	Value	Where it is set
observation size	15	<code>observation_space</code> in <code>TricycleEnvCfg</code>
action size	2 (throttle, steer)	<code>action_space</code>
physics step	10 ms (<code>dt = 1/100</code>)	<code>SimulationCfg</code>
control step	20 ms (<code>decimation = 2</code>)	<code>TricycleEnvCfg</code>
episode length	2 s = 100 control steps	<code>episode_length_s</code>
reward per step	forward displacement, clipped to ± 0.1 m	<code>_get_rewards</code>
episode return	metres driven forward in 2 s	sum of the above
rollout length	50 steps per env per iteration	<code>num_steps_per_env</code>
iterations	300	<code>max_iterations</code>
checkpoint every	50 iterations	<code>save_interval</code>
networks	15 $\rightarrow$ 64 $\rightarrow$ 64 $\rightarrow$ 2 and 15 $\rightarrow$ 64 $\rightarrow$ 64 $\rightarrow$ 1	<code>policy cfg</code>

Because the reward is displacement, the “mean episode reward” printed by `rs1_rl` is directly readable: a value of 1.4 means the average car drove 1.4 m forward in its 2 s episode. The runbook’s success criterion (reward climbs past ~ 2 m without diverging) is expressed in the same unit.

Chapter 3

From isaacsim train to a checkpoint

This chapter follows one training run from the command line to the saved policy, naming the Isaac Lab function that performs each step. Nothing here is specific to the tricycle except the names of our classes; every Isaac Lab task on the develop branch goes through the same path.

3.1 The command

```
.\isaacsim.bat train --rl_library rsl_rl --task Luna-Tricycle-Direct --num_envs 4
```

- `isaacsim.bat` activates the venv’s Python and runs the `isaacsim` console script, i.e. the function `cli()` in `source/isaacsim/isaacsim/cli/__init__.py`.
- `train` is the sub-command. Others are `play`, `zero_agent`, `random_agent`, `benchmark`.
- `--rl_library rsl_rl` picks the learning library backend.
- `--task Luna-Tricycle-Direct` is the Gymnasium id registered by `tricycle/__init__.py`. There is no `-v0` suffix.
- `--num_envs 4` overrides `scene.num_envs` from the config.
- Optional: `--viz newton` opens the Newton OpenGL window, `--max_iterations N` overrides the runner config, `physics=newton_mjwarp` is a Hydra “typed selector” that swaps the physics section for a preset (not needed here because our `__post_init__` builds its own `NewtonCfg`).

3.2 The whole path at a glance



Figure 3.1: The training path. Blue boxes are code in this repository; grey boxes are Isaac Lab or rsl_rl.

3.3 Step 1: the CLI finds our package

cli() checks whether the first argument is one of its sub-commands. If it is, it first calls `_load_external_tasks()`:

```

_TASK_ENTRY_POINT_GROUP = "isaacsim.tasks"

def _load_external_tasks() -> None:
    for entry_point in importlib.metadata.entry_points(group=_TASK_ENTRY_POINT_GROUP
    ↪ ):
        entry_point.load()

```

Isaac Lab: entry points

An *entry point* is a line of metadata that a package writes into the Python environment when it is installed. Isaac Lab defines the group name `isaacsim.tasks`; any installed package that lists something under that group gets imported by the CLI before the task name is looked up. Our `pyproject.toml` lists `luna_hifi_tasks = "luna_hifi_tasks"`, so `entry_point.load()` performs `import luna_hifi_tasks`. Entry points are read from the installed metadata, not from the source file, which is why editing `pyproject.toml` requires re-running `pip install -e`.

Importing `luna_hifi_tasks` runs the chain described in Section 1.2: the package `__init__` imports `tricycle`, whose `__init__` calls `gym.register`. After this, Gymnasium's global registry maps the string `Luna-Tricycle-Direct` to three dotted paths: the environment class, the environment config class and the `rsl_rl` agent config class (Chapter 8).

`cli()` then calls `train()`, which imports `isaacsim_rl.entrypoints.run_train_cli`. That function reads `-rl_library` and dispatches to `isaacsim_rl/entrypoints/backends/train_rsl_rl.py`.

3.4 Step 2: configs are built and overridden

The backend uses the `hydra_task_config` decorator from `source/isaacsim_tasks/isaacsim_tasks/utils/hydra.py`. Its `register_task` function does the following, in order:

1. `load_cfg_from_registry(task, "env_cfg_entry_point")` looks up the registered string `"...tricycle_env_cfg:TricycleEnvCfg"`, imports that module and *instantiates the class*. Instantiating a configclass runs its `__post_init__`, so at this moment our `TricycleEnvCfg.__post_init__` builds the whole `NewtonCfg` (Chapter 10). The same is done for the agent config.
2. Presets are collected and any `physics=NAME` or `presets=...` selectors on the command line are applied with *replace* semantics: the whole config section is swapped, not merged.
3. If this is a play run, `env_cfg.play_mode()` is called (Section 3.9).
4. Remaining `env.xxx=value` / `agent.xxx=value` scalar overrides are set with `setattr`, and anything else is handed to Hydra proper.

Separately, the backend's argument parser handles the classic flags. In `isaacsim_rl/entrypoints/common.py`, `apply_env_overrides` does

```
if getattr(args_cli, "num_envs", None) is not None:
    env_cfg.scene.num_envs = args_cli.num_envs
```

Watch out: `-num_envs` arrives after `__post_init__`

The config object is fully built, `__post_init__` included, before the command-line `-num_envs` is written into it. Anything you computed inside `__post_init__` from `self.scene.num_envs` used the *class default* (8), not the number you typed. This is why the MPM sparse-grid capacities in `tricycle_env_cfg.py` are fixed numbers that must be raised by hand when you train with more envs (Chapter 4).

3.5 Step 3: the environment is built

`gym.make("Luna-Tricycle-Direct", cfg=env_cfg, ...)` instantiates `TricycleEnv`, whose `__init__` immediately calls `DirectRLEnv.__init__` (`source/isaacsim/isaacsim/envs/direct_rl_env.py`). That constructor does a great deal, in this order:

1. `cfg.validate()`: every MISSING field must have been filled in (`decimation`, `episode_length_s`, `scene`, `observation_space`, `action_space` are all required).
2. Seeds torch/numpy if `cfg.seed` is set (the runner sets it).
3. Creates the `SimulationContext` from `cfg.sim`. This is the object that owns the physics backend; because our `cfg.sim.physics` is a `NewtonCfg`, the Newton manager is chosen (Chapter 4).
4. `self.scene = InteractiveScene(self.cfg.scene)`: walks every field of `TricycleSceneCfg`, spawns the prims on the USD stage (ground plane, light, the car USD, the hidden slab, the soil particle spawner), and creates the Python asset objects (`Articulation`, `RigidBody`, `MPMObject`) that will wrap them.
5. `self._setup_scene()`: our hook. In this repository it only fetches the already-created assets by name.
6. `self.sim.reset()`: starts physics. For Newton this is where the USD stage is read into a `Newton ModelBuilder`, replicated once per env, finalised into a `Model`, and where the coupled solver (rigid + MPM) is constructed. After this call, tensors like `car.data.root_pos_w` are live.
7. `self.scene.update(dt)` fills the data buffers once.
8. Allocates the per-env bookkeeping tensors: `episode_length_buf`, `reset_terminated`, `reset_time_outs`, `reset_buf`, and builds the Gymnasium spaces from `observation_space` and `action_space` (an int becomes a `Box(-inf, inf, (n,))`).

Only when `super().__init__` returns does the rest of `TricycleEnv.__init__` run: looking up joint indices and allocating the action and previous-position buffers (Chapter 11).

Isaac Lab: InteractiveScene and asset objects

`InteractiveScene` (`source/isaacsim/isaacsim/scene/interactive_scene.py`) turns a scene *config* into scene *objects*. Each attribute of the scene `cfg` whose type is an asset `cfg` (`ArticulationCfg`, `RigidBodyCfg`, `MPMObjectCfg`, `AssetBaseCfg`) becomes an entry in the scene, retrievable with `scene["name"]`. The scene also owns `env_origins`, an $(N, 3)$ tensor with the world position of each env's origin, spaced `env_spacing` metres apart on a grid.

3.6 Step 4: the wrapper

`rsl_rl` expects a specific interface (a `VecEnv`) rather than the Gymnasium one. `RslRlVecEnvWrapper` (`source/isaacsim_rl/isaacsim_rl/rsl_rl/vecenv_wrapper.py`) adapts it:

- `get_observations()` calls our `_get_observations()` and returns the dictionary as a `TensorDict`. Our env returns a dict with a single group, "policy"; `rsl_rl` maps that group to both the actor and the critic when no `obs_groups` are configured.
- `step(actions)` optionally clips actions to `clip_actions`, calls `DirectRLEnv.step`, merges `terminated` | `truncated` into a single `done` tensor, and stores `truncated` as `extras["time_outs"]` so PPO can bootstrap (Section 2.6).
- Exposes `num_envs`, `num_actions`, `episode_length_buf` and `device`.

3.7 Step 5: the runner

`train_rsl_rl.py` builds the log directory `logs/rsl_rl/<experiment_name>/<YYYY-MM-DD_HH-MM-SS>`, dumps both configs into it as YAML (`params/env.yaml`, `params/agent.yaml`), then

creates the runner and starts learning:

```
runner = OnPolicyRunner(env, agent_cfg.to_dict(), log_dir=log_dir, device=agent_cfg.
    ↪ device)
runner.learn(num_learning_iterations=agent_cfg.max_iterations)
```

Inside learn (rsl_rl/runners/on_policy_runner.py) every iteration is:

```
for _ in range(num_steps_per_env):          # 50 in our config
    actions = self.alg.act(obs)              # sample from the Gaussian policy
    obs, rewards, dones, extras = self.env.step(actions)
    self.alg.process_env_step(obs, rewards, dones, extras) # store + handle time_
    ↪ outs
    self.alg.compute_returns(obs)             # GAE
    loss_dict = self.alg.update()            # PPO epochs / mini-batches
    # log, and every save_interval iterations: torch.save(..., f"model_{it}.pt")
```

TensorBoard event files are written next to the checkpoints, so `tensorboard -logdir logs/rsl_rl/tricycle` shows the curves.

3.8 One environment step, in detail

`DirectRLEnv.step(action)` is the heart of the whole system and the order of operations inside it decides what your overridden methods may assume. Figure 3.2 shows it for our `decimation = 2`.

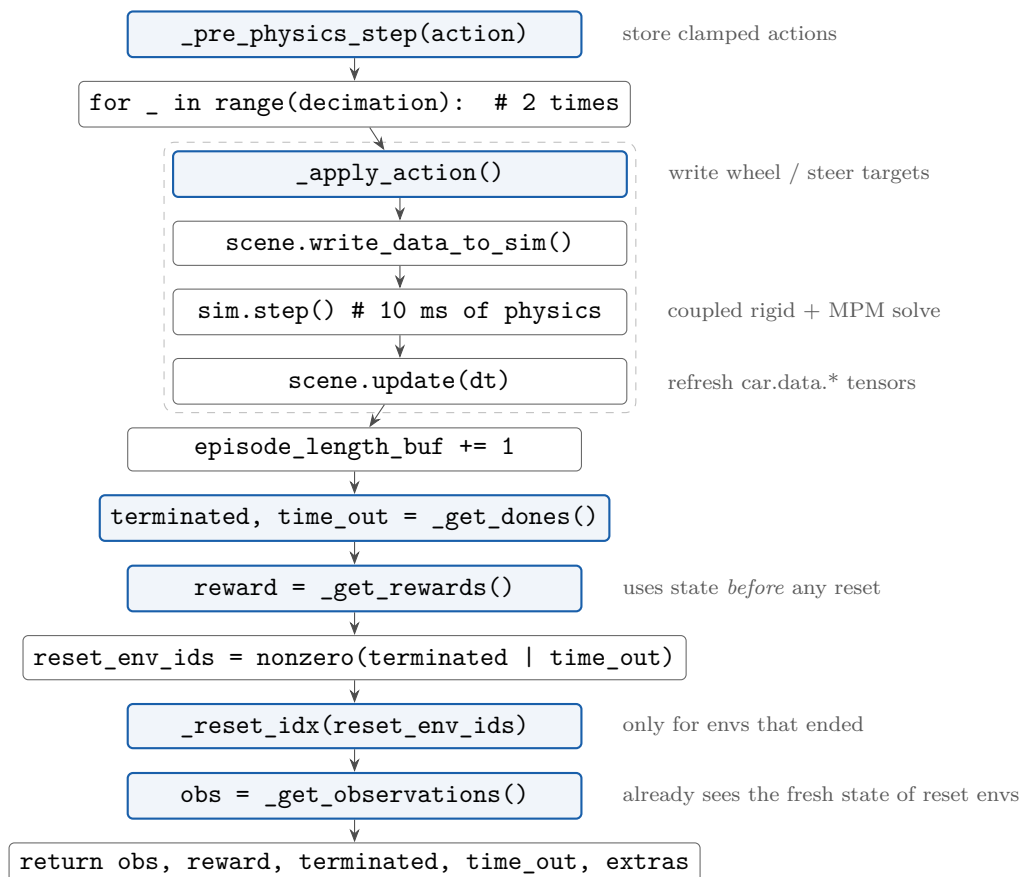


Figure 3.2: Order of operations in `DirectRLEnv.step`. Blue: methods implemented in `tricycle_env.py`. The dashed group runs decimation times per call.

Three consequences that the tricycle code relies on:

- **Rewards are computed before resets.** `_get_rewards` may compare the current car position with the position stored last step (`_prev_x`); at that moment no env has been teleported yet.
- **Observations are computed after resets.** The observation returned for an env that just ended already describes the *new* episode. `rsl_rl` knows the episode boundary from `done`s, so this is the expected behaviour.
- **`_apply_action` runs every physics sub-step, `_pre_physics_step` once per control step.** Cheap work (writing targets) belongs in the former, decoding the action in the latter.

Isaac Lab: the Newton fast path for decimation

After `sim.reset()`, `DirectRLEnv` asks the physics manager whether it can run the whole decimation loop itself (`physics_manager.set_decimation(...) / handles_decimation()`). When it can, the loop in Figure 3.2 collapses into a single `sim.step` that internally performs decimation physics steps under one CUDA graph, and `_apply_action()` is called once. The observable behaviour for our task is identical because `_apply_action` writes the same targets each time.

3.9 Play

```
.\isaacsim.bat play --rl_library rsl_rl --task Luna-Tricycle-Direct --num_envs 1 --
    ↪ checkpoint latest --viz newton
```

`play` follows the same path with two differences. First, `hydra_task_config(..., play_mode=True)` calls `env_cfg.play_mode()` after the presets are resolved and before scalar overrides, so our override in `TricycleEnvCfg` caps the env count and switches on particle rendering (Chapter 10); `-num_envs 1` then still wins because it is applied afterwards. Second, instead of learning, `play_rsl_rl.py` resolves the checkpoint (`latest` means the newest run directory under `logs/rsl_rl/tricycle` and its highest-numbered `model_*.pt`), loads it into the runner, takes `runner.get_inference_policy()` (the deterministic mean action), exports the network to `exported/policy.pt` and `policy.onnx`, and loops `obs, _, done, _ = env.step(policy(obs))` until the window closes.

`-viz newton` attaches the Newton OpenGL visualiser. It draws rigid shapes by default; soil particles are drawn only because our `play_mode()` sets `show_particles=True`.

Chapter 4

The physics stack

The tricycle runs on **Newton**, NVIDIA’s GPU physics engine built on Warp. Isaac Lab talks to Newton through a *physics manager*, and the tricycle uses two of Newton’s solvers at once: a rigid-body solver for the car and an implicit Material Point Method (MPM) solver for the soil. This chapter explains the pieces that `tricycle_env_cfg.py` assembles, so that the config in Chapter 10 reads as a list of decisions rather than magic numbers.

4.1 Layers

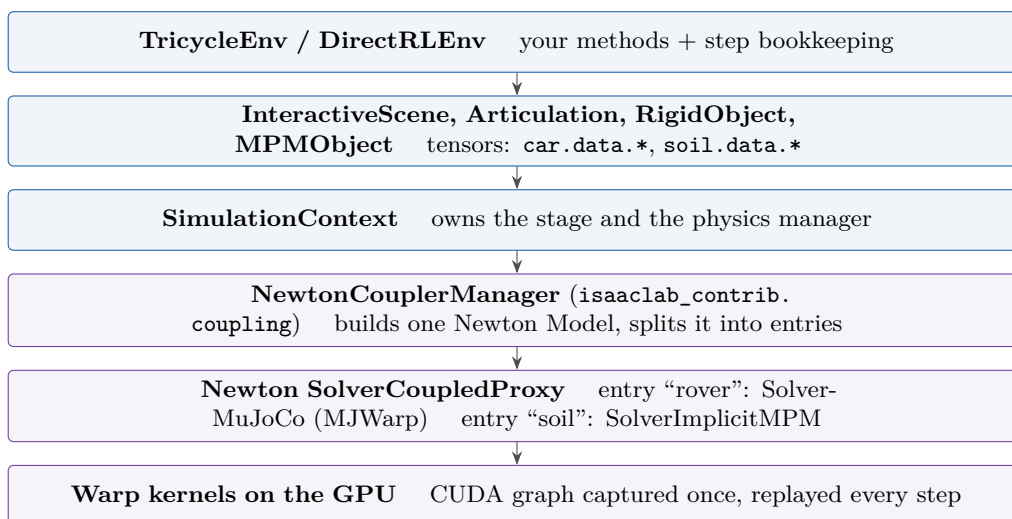


Figure 4.1: From environment code to GPU kernels. Blue layers are Isaac Lab; purple layers are Newton.

Isaac Lab: physics managers

Isaac Lab develop can drive several physics backends (PhysX, Newton, OmniPhysX). `SimulationCfg.physics` selects one by holding a `PhysicsCfg` subclass; `NewtonCfg` (source/isaacrab_newton/isaacrab_newton/physics/newton_manager_cfg.py) selects Newton. Inside `NewtonCfg`, `solver_cfg` selects *which* Newton solver manager runs the model: `MJWarpSolverCfg`, `MPMSolverCfg`, `XPBDSolverCfg`, ..., or, as here, a `CouplerProxyCfg` from `isaacrab_contrib.coupling` that nests two of them. `NewtonCfg.__post_init__` copies the manager class from `solver_cfg.class_type` so you never set it yourself.

4.2 The rigid-body side: MuJoCo-Warp

The car is an *articulation*: a tree of rigid bodies (chassis, fork, three wheels) connected by joints (one free joint for the chassis, one steering hinge, three wheel hinges). Newton simulates it with **MJWarp**, a GPU port of MuJoCo’s dynamics. Isaac Lab wraps it as **MJWarpSolverCfg** (`physics/mjwarp_manager_cfg.py`). The four options the config sets:

use_mujoco_contacts=False MuJoCo normally detects and resolves its own contacts. Setting this to **False** hands collision detection to Newton’s own *collision pipeline*, which is required whenever a Newton `collision_cfg` is also given (the two are mutually exclusive) and is what lets the coupler share contacts between solvers.

integrator="implicitfast" MuJoCo’s implicit-in-velocity integrator. It damps stiff joint drives and contacts far better than plain Euler at the same step size, which matters with a 10 ms step and velocity-driven wheels.

njmax=128 maximum number of constraint rows per env (joint limits, contacts, friction cones). Too small and contacts are dropped with a warning; too large only costs memory.

nconmax=128 maximum number of contact points per env.

Isaac Lab: implicit actuators and use_newton_actuators

Our joints use `ImplicitActuatorCfg`. “Implicit” means the drive is a PD controller *inside the physics solver*: you give a position target (steering) or a velocity target (rear wheels) plus **stiffness** and **damping**, and the solver computes the torque as part of its implicit step. `effort_limit_sim` and `velocity_limit_sim` are written into the simulator as hard limits. `sim.use_newton_actuators = True` asks Newton to author and execute these drives natively rather than through the host-side adapter used for PhysX.

4.3 The soil side: implicit MPM

Newton: what MPM is

The Material Point Method represents a continuum (soil, snow, mud) as a cloud of **particles** that carry mass, velocity and deformation history, plus a background **grid** on which forces are solved each step. Every step: particle quantities are transferred to the grid (“P2G”), the momentum equations with the material’s constitutive law are solved on the grid, and grid velocities are transferred back to move the particles (“G2P”). Rigid bodies enter as *colliders*: signed-distance boundaries the grid velocities must respect. Newton’s solver is *implicit*: it solves the yield condition as an optimisation problem each step, which allows large steps (our 10 ms) without exploding.

Isaac Lab exposes it as `MPMSolverCfg` (`physics/mpm_manager_cfg.py`). The fields the tricycle sets, grouped:

Grid. `voxel_size=0.05` is the grid spacing (5 cm). `grid_type="sparse"` allocates grid cells only where particles are. `grid_padding=0` adds no empty margin. `separate_worlds=True` gives every env its own grid so envs cannot touch each other’s soil and particle arrays are laid out contiguously per env.

Sparse-grid capacities. `max_active_cell_count`, `max_leaf_node_count`, `max_lower_node_count`, `max_upper_node_count` pre-allocate the sparse tree (leaf → lower → upper nodes) so that the CUDA graph can be captured once. They are *totals across all envs*, not per env. Newton

requires `upper <= lower <= leaf <= active`. The committed values $2^{14}/2^{13}/2^{12}/2^{10}$ were found by hand for 4 envs on 6 GB; a “capacity exceeded” error names the tier to raise, and a “Failed to create volume” plus CUDA error 700 means the caps were raised too far for the memory available.

Numerics. `max_iterations=24` and `tolerance=1e-4` bound the per-step rheology solve. `solver="auto"` lets Newton pick Gauss–Seidel for the Q1 velocity basis. `warmstart_mode="auto"` reuses last step’s solution as a starting guess. `strain_basis="P0"` (piecewise constant strain per cell), `velocity_basis="Q1"` (trilinear velocities) and `transfer_scheme="apic"` (affine particle-in-cell transfers, less numerical damping than PIC) are the standard granular preset also used by Isaac Lab’s own `ur10_particle_push` task.

Colliders. `collider_basis="pic27"` samples collider velocities with a 27-point stencil; `collider_velocity_mode="forward"` uses the colliders’ velocities at the start of the step. `project_outside_colliders=False` is mandatory on a coupled entry: it disables a particle-level push-out pass that would fight the coupler.

The material. Particles carry an `MPMParticleMaterialCfg` (`sim/spawners/mpm/mpm_cfg.py`). Its fields: `density` [kg/m^3], `young_modulus` [Pa], `poisson_ratio`, `viscosity` [$\text{Pa}\cdot\text{s}$], `friction` (dimensionless, $\approx \tan \phi$ of the internal friction angle), `damping` [s], `yield_pressure` [Pa], `tensile_yield_ratio`, `yield_stress` [Pa] (cohesion), `hardening`, `dilatancy`. The yield surface is

$$\tau_{\max}(p) = \text{yield_stress} + \text{friction} \cdot (p - p_{\min}),$$

so shear strength grows with pressure p like dry sand. The tricycle soil sets `density=1800`, `friction=0.84` ($= \tan 40^\circ$) and `yield_pressure=1e12` (effectively incompressible), leaving the very stiff default `young_modulus` so the strip behaves like packed ground rather than a sandbox.

The particles. `MPMGridCfg` fills an axis-aligned box with a lattice. Its resolution per axis is $\lceil \text{particles_per_cell} \cdot \text{extent}/\text{voxel_size} \rceil$, so the $1.5 \times 0.6 \times 0.06$ m strip at 5 cm voxels gives $30 \times 12 \times 2 = 720$ particles per env, each placed at a cell centre (`particle_placement="cell_center"`) with `mass = cell volume  $\times$  density`.

Why: the soil needs its own floor

The MPM solver only sees colliders that belong to *its* coupler entry. The world ground plane belongs to the rigid entry, so to the soil it does not exist and particles would fall forever. `tricycle_env_cfg.py` therefore adds `MPMGround`, an invisible kinematic slab just below the strip, and lists it as the MPM entry’s only body.

4.4 Coupling the two solvers

Newton: one Newton model, two solvers

Newton’s experimental *coupled solver* takes a single `Model` and a list of **entries**. Each entry owns a subset of bodies, joints, shapes and particles (a `ModelView`) and runs its own solver on that subset. **Proxy mappings** expose selected bodies of one entry inside another as *virtual proxies*: massive ghost bodies whose pose is copied from the source each step and whose accumulated contact impulses are sent back as feedback forces. Newton calls this *lagged-impulse* coupling because the feedback is applied one step late.

Isaac Lab wraps this as `CouplerProxyCfg` (`source/isaacsim/isaacsim/isaacsim/coupling/coupler_cfg.py`):

- **entries:** a list of `CouplerEntryCfg`. Each has a `name`, a `solver_cfg`, a list of `bodies` (regexes matched against full Newton body labels, descendants included), `all_particles`, `include_static_shapes` (whether the entry owns shapes with no body, such as the ground plane), `include_child_joints`, `substeps` (how many equal sub-steps the entry takes inside one coupled step) and `in_place` (step into the same state buffer; required `True` for MPM).
- **proxies:** a list of `CouplerProxyMappingCfg` with `source`, `destination`, `bodies`, `mode` ("lagged" or "staggered"), `mass_scale` and `collision_pipeline`.
- **iterations:** proxy relaxation passes per coupled step.

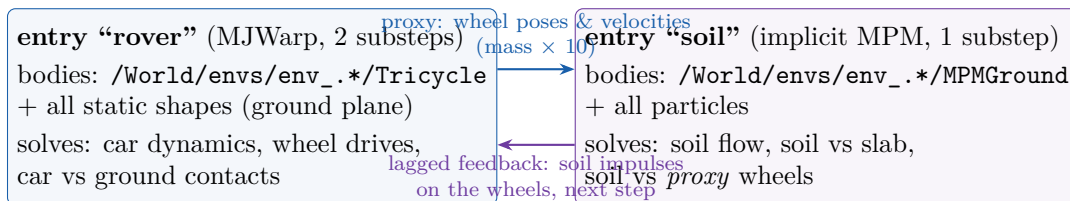


Figure 4.2: The two coupler entries and the one proxy mapping in `TricycleEnvCfg.__post_init__`. Only the three wheel bodies are mirrored into the soil solver.

What happens each 10 ms physics step: the rigid entry advances the car by two 5 ms sub-steps using last step’s soil feedback; the wheels’ new poses and velocities are copied onto their proxies inside the MPM view; the MPM entry advances the soil one 10 ms step, colliding particles against the proxy wheels and the slab; the impulses the particles exerted on the proxies are stored and applied to the real wheels during the *next* rigid step.

Why: `mass_scale` is the stability knob

A proxy body is given the source body’s mass times `mass_scale`. A heavy proxy barely moves when soil pushes on it, so the soil solve stays well-posed and the feedback impulses are smooth; the real wheel then receives those impulses one step late. If the scale is too small the lagged feedback can oscillate (the car “chatters” on the strip); too large and the soil feels the wheel as an immovable wall. The tricycle uses 10 through `TricycleEnvCfg.proxy_mass_scale`.

Watch out: rules the coupler enforces

- An MPM entry must have `in_place=True`; the coupler raises `ValueError` otherwise (`coupler.py`, `_validate_config`).
- A proxy *source* entry may not be `in_place` in lagged mode (Newton, `solver_coupled_proxy.py`). The rigid entry is not, so this is satisfied.
- `project_outside_colliders` must be `False` on a coupled MPM entry.
- Newton’s proxy coupler supports at most two entries.

4.5 The collision pipeline and `soft_contact_max=0`

With `use_mujoco_contacts=False`, rigid contacts (car against ground plane) come from Newton’s `CollisionPipeline`, configured by `NewtonCollisionPipelineCfg` (`physics/newton_collision_cfg.py`). The only field the tricycle sets is `soft_contact_max=0`. “Soft contacts” are particle-versus-shape contacts generated by the generic pipeline; their default allocation is `shape_count * particle_count`, which for thousands of particles is a huge buffer, and the MPM solver handles particle contacts itself anyway. Setting the maximum to zero disables that allocation.

4.6 Time stepping and CUDA graphs

`SimulationCfg.dt = 1/100` is the physics step; `NewtonCfg.num_substeps=1` means one coupled solve per step (the rigid entry's own `substeps=2` happens inside it). `use_cuda_graph=True` records the whole step as a CUDA graph the first time and replays it afterwards, which is where most of the speed comes from; it also explains why buffer sizes (grid capacities, `njmax`) must be fixed up front. `decimation=2` in the env cfg means the policy acts every second physics step, i.e. at 50 Hz.

4.7 Why only a handful of envs fit

Each env carries 720 particles plus its own sparse grid; the grid tiers are shared caps. The GPU work per step is dominated by the number of *active grid cells* times envs, and the 6 GB card runs out of memory well before the GPU runs out of compute. The runbook's numbers (4 envs, 90–160 steps/s) are the practical ceiling for this card; the same configuration on a large GPU simply needs the four caps raised in proportion to `-num_envs`.

Chapter 5

From MJCF to USD: the car asset

The car is described twice in the repository: once as MJCF text inside `tricycle.py`, and once as a set of USD files under `assets/tricycle/`. Only the USD is loaded at run time. This chapter explains how one becomes the other and what each USD layer contains, so that Chapters 9 and 13 can go line by line.

5.1 Why two descriptions

MJCF (MuJoCo’s XML format) is compact and human-writable: a body, a joint, a geom, done. Isaac Lab’s Newton backend, however, builds its physics model from the *USD stage* that `InteractiveScene` spawns. Isaac Lab can convert MJCF to USD, but the importer it uses is a Kit extension that only exists inside a running Isaac Sim application; the `isaacsim train` command never starts Kit. So the conversion is done once, offline, with `scripts/tools/convert_mjcf.py`, and the result is committed.

Watch out: stale assets

Nothing checks that `assets/tricycle/` matches `tricycle.py`. If you edit the MJCF and forget to reconvert, training silently uses the old car. The runbook’s “Convert the tricycle asset” step is the fix, and `tricycle.py` now says so in its header. For this revision the committed USD was regenerated to match the new body.

5.2 Anatomy of the MJCF

```
<mujoco model="tricycle">
  <compiler .../>           units and options for MuJoCo's model compiler
  <default> ... </default> attributes every geom / joint inherits unless overridden
  <worldbody>
    <body name="chassis">    a rigid body; pos is relative to its parent
      <freejoint/>           6 degrees of freedom: the chassis floats
      <geom .../>           a shape with mass, attached to the body
      <body name="fork">    a child body; its pos is relative to chassis
        <joint .../>        1 degree of freedom between fork and chassis
      ...
```

Bodies form a tree. Each body’s pose is given relative to its parent, each joint sits between a body and its parent, and each geom belongs to exactly one body. Mass and inertia of a body are the sums over its geoms (MuJoCo computes the inertia tensor of a box or sphere from its dimensions and the given mass). That is why every body in `tricycle.py` has at least one geom with a mass: a body with zero inertia makes the solver divide by zero.

5.3 The converter

`convert_mjcf.py` wraps `isaacsim.converters.MjcfConverter`. On the develop branch this converter drives the *MuJoCo USD Converter* (the `customLayerData` of every output file says "MuJoCo USD Converter v0.2.0"), which compiles the MJCF with MuJoCo's own compiler and then writes USD prims for what it finds. Isaac Lab then runs two post-processing passes recorded in `assets/config.yaml`: `run_asset_transformer` adds the Isaac robot schemas (`IsaacRobotAPI`, `IsaacLinkAPI`, `IsaacJointAPI`), and `run_multi_physics_conversion` splits the physics into selectable variants (`physics`, `mujoco`, `physx`, `none`).

The output is a small tree of `.usda` files (USD's text format):

File	Contains
<code>tricycle.usda</code>	the entry point: references <code>base.usda</code> , defines the <code>Physics</code> variant set
<code>payloads/base.usda</code>	the geometry: one <code>Xform</code> per body, one <code>Cube/Sphere/Capsule</code> per geom
<code>payloads/robot.usda</code>	Isaac robot/link/joint tags, sublayered into <code>base.usda</code>
<code>payloads/Physics/physics.usda</code>	rigid-body, articulation-root, collision and mass schemas; the joints
<code>payloads/Physics/mujoco.usda</code>	MuJoCo-only extras (<code>MjcJointAPI</code> with <code>mjc:damping</code>)
<code>payloads/Physics/physx.usda</code>	PhysX-only extras (<code>PhysxJointAPI</code>); unused with Newton

Isaac Lab: USD in five sentences

A USD *stage* is a tree of *prims* (nodes) each with a type (`Xform`, `Cube`, `PhysicsRevoluteJoint`, ...) and attributes. *API schemas* such as `PhysicsRigidBodyAPI` are tags that can be applied to a prim to give it extra attributes and meaning. A stage is *composed* from several files: `def` creates a prim, `over` adds to a prim defined elsewhere, `references` and `payload` pull in other files, `subLayers` stack files. A *variant set* is a switch that selects which of several alternative sub-trees is active; Isaac Lab leaves the `Physics` variant on "physics", so the `mujoco` and `physx` payloads are not loaded. Newton reads the composed result: bodies from `PhysicsRigidBodyAPI` prims, shapes from prims with `PhysicsCollisionAPI`, masses from `PhysicsMassAPI`, joints from `Physics*Joint` prims.

5.4 How MJCF names become prim paths

The converter keeps the body tree and the body names:

MJCF body tree	USD prim (inside the asset)
<code>chassis</code>	<code>/tricycle/Geometry/chassis</code>
<code>fork</code>	<code>/tricycle/Geometry/chassis/fork</code>
<code>front_wheel_body</code>	<code>/tricycle/Geometry/chassis/fork/front_wheel_body</code>
<code>rear_left_body</code>	<code>/tricycle/Geometry/chassis/rear_left_body</code>
<code>rear_right_body</code>	<code>/tricycle/Geometry/chassis/rear_right_body</code>
joint <code>"steer"</code>	<code>/tricycle/Geometry/chassis/fork/steer</code>
joint <code>"front_wheel"</code>	<code>/tricycle/Geometry/chassis/fork/front_wheel_body/</code>
$\hookrightarrow$ <code>front_wheel</code>	
joint <code>"rear_left"</code>	<code>/tricycle/Geometry/chassis/rear_left_body/rear_</code>
$\hookrightarrow$ <code>left</code>	

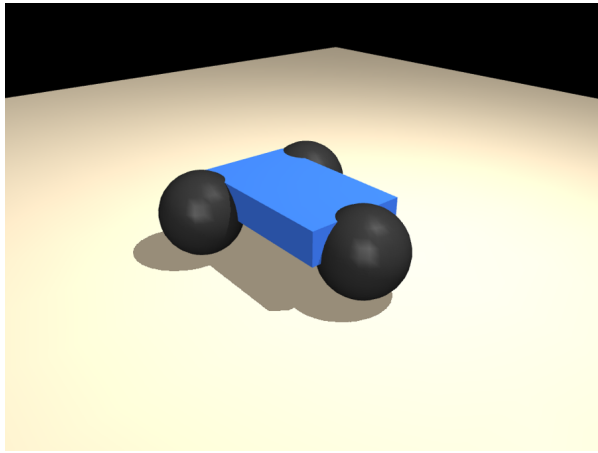
When `InteractiveScene` spawns the asset at `prim_path="{ENV_REGEX_NS}/Tricycle"`, the asset root `/tricycle` is placed at `/World/envs/env_0/Tricycle`, `/World/envs/env_1/Tricycle`, and so on. The full Newton body label of the left rear wheel in env 0 is therefore `/World/envs/env_0/Tricycle/Geometry/chassis/rear_left_body`. This is where two conventions in the env cfg come from:

- Scene configs use the macro `{ENV_REGEX_NS}`, which Isaac Lab expands to `/World/envs/env_.*`.
- Coupler bodies lists are plain regexes matched by Newton, so they spell the expansion out: `/World/envs/env_.* /Tricycle/Geometry/chassis/.*_body` matches all three wheel bodies (the `.*` also swallows `fork/`), but not the chassis or the fork.

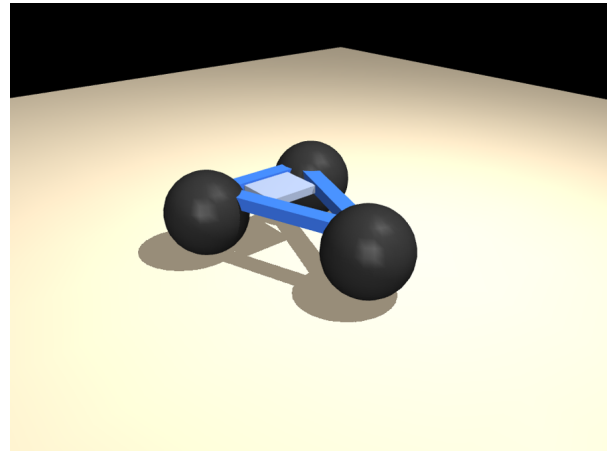
Joint *names* survive too: `Articulation.find_joints(["steer"])` works because the USD joint prim is called `steer`.

5.5 What the new body looks like

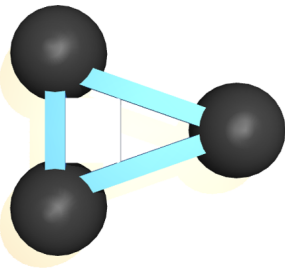
The original chassis was a single $40 \times 24 \times 8$ cm box with the front wheel poking out of its nose. It has been replaced by a triangular frame: two side rails running from the steering head back to the rear corners, a rear crossbar, and a flat deck inside the rear half. All four are box geoms; the total chassis mass (5.0 kg), the wheel positions, radii and masses, and every joint and body name are unchanged, so the env config and the coupler mapping did not have to change.



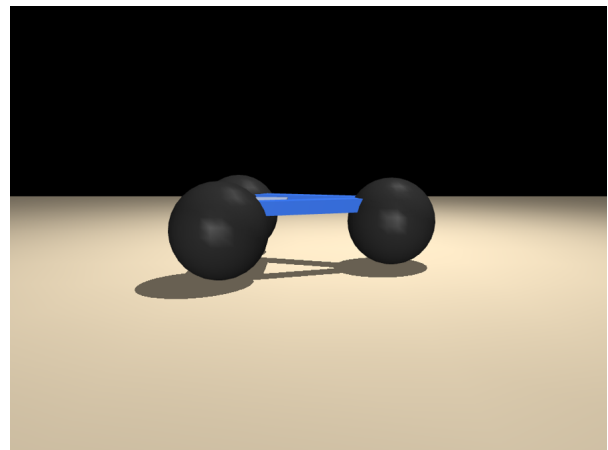
(a) Before: box chassis.



(b) After: triangular frame with deck.



(c) After, plan view. Apex forward (right), rear wheels at the base.



(d) After, side view. The frame sits at 14 cm, wheel centres at 10 cm.

Figure 5.1: The tricycle as compiled by MuJoCo from `tricycle.py` (rendered with MuJoCo's own renderer, not Isaac Lab).

Because the wheels are 10 cm spheres and the frame sits at the chassis height of 14 cm, the frame's three corners disappear inside the wheels: the rails run into the rear wheels and the apex sits inside

the front wheel. That is intentional; the corners of a real tricycle frame are also hidden by its hubs.

5.6 Regenerating the USD

Whenever `tricycle.py` changes, run the two commands in `RUNBOOK.md`, “Convert the tricycle asset”: the first writes the MJCF to a temp file with `write_mjcf()`, the second converts it. The converter writes into `assets/tricycle/tricycle.usda` regardless of the exact output filename given, and updates `assets/config.yaml` and `assets/.asset_hash`. Isaac Lab uses the hash to skip conversion when neither the MJCF nor the converter settings changed (`force_usd_conversion: true` in the committed config always reconverts).

Part II

Every file, every line

Chapter 6

pyproject.toml

Role: tells `pip` how to install the package and tells Isaac Lab that the package contains tasks. It is read once, at install time (Section 1.2). 17 lines.

```
1 [project]
2 name = "luna_hifi_tasks"
3 version = "0.1.0"
4 description = "Isaac Lab tasks for rover excavation on Newton MPM soil"
5 requires-python = ">=3.11"
6 dependencies = []
7
8 [project.entry-points."isaac.lab.tasks"]
9 "luna_hifi_tasks" = "luna_hifi_tasks"
10
11 [build-system]
12 requires = ["setuptools>=64"]
13 build-backend = "setuptools.build_meta"
14
15 [tool.setuptools.packages.find]
16 where = ["."]
17 include = ["luna_hifi_tasks*"]
```

Line 1 Opens the `[project]` table, the standard place (PEP 621) for a package's metadata.

Line 2 The distribution name. This is what `pip list` shows and what the installed metadata folder is called (`luna_hifi_tasks-0.1.0.dist-info`). It happens to equal the import name, but the two are independent.

Line 3 Version string; not used by anything in Isaac Lab, but required by `pip`.

Line 4 Free-text description shown by `pip show`.

Line 5 Refuses to install on Python older than 3.11. Newton 1.5 uses the `X | None` union syntax in type annotations at run time, which Python 3.10 rejects (RUNBOOK section 1). The runbook goes further and uses 3.12.

Line 6 No dependencies are declared, deliberately. Isaac Lab, Newton, torch, warp and gymnasium all come from the Isaac Lab venv; declaring them here would make `pip` try to resolve versions it must not touch.

Line 7 Blank separator.

Line 8 Opens the entry-point table for the group `isaac.lab.tasks`. The quotes are needed because the group name contains a dot. This is the single line that makes the package discoverable by the `isaac.lab` command (Section 3.3).

Line 9 One entry point: name `luna_hifi_tasks`, target the module `luna_hifi_tasks`. When Isaac Lab calls `entry_point.load()` on it, Python imports that module, which triggers the task registration. The target could have been a sub-module or an object; the whole package is used so that adding `from . import excavation` later registers the second task without touching this file.

Line 10 Blank.

Line 11 Opens the build-system table: which tool turns this folder into an installable package.

Line 12 Requires `setuptools` 64 or newer, the first version with PEP 660 editable installs. That is what makes `pip install -e` work without a `setup.py`.

Line 13 The build backend entry point inside `setuptools`.

Line 14 Blank.

Line 15 Configures `setuptools`' automatic package discovery.

Line 16 Search for packages starting in the repository root.

Line 17 Only take folders whose name starts with `luna_hifi_tasks`. This excludes `assets`, `docs` and anything else at the root. A folder only counts as a package if it contains `__init__.py`; a sub-folder without one is silently skipped, which is the missing-`__init__.py` failure mode in the runbook. Adding the file is the whole fix; see the box below on why no reinstall is needed for it.

Isaac Lab: editable installs, and when a reinstall is needed

`pip install -e <folder>` does not copy the code. With a modern `setuptools` (PEP 660) it writes two things into the venv's `site-packages`: a `.pth` file that installs an import finder, and the distribution metadata in `luna_hifi_tasks-0.1.0.dist-info`. The finder holds a single mapping, the top-level name `luna_hifi_tasks` pointing at your source folder; every module below it is resolved off the filesystem at import time.

So the source tree is read live, and *one* kind of change needs a reinstall: editing `pyproject.toml`. Entry points and the package list are frozen into `dist-info` when you install, so a new or renamed entry point stays invisible until `pip install -e` runs again. Everything else takes effect on the next run: editing any `.py` file, adding a new subpackage, and adding an `__init__.py` to a folder that lacked one.

The `luna_hifi_tasks.egg-info/` folder that appears in the repository root is a leftover of the build, not the metadata anything reads; deleting it does not affect task discovery. All four behaviours were checked directly on this package in a clean virtual environment.

Chapter 7

luna_hifi_tasks/__init__.py

Role: the package root. Importing it is what the Isaac Lab entry point does at start-up; its only job is to import the sub-packages that register tasks. 1 line.

```
1 from . import tricycle # noqa: F401
```

Line 1 A relative import of the `tricycle` sub-package. Python executes `luna_hifi_tasks/tricycle/__init__.py` as a side effect, and that file registers `Luna-Tricycle-Direct` with Gymnasium (Chapter 8). The trailing `# noqa: F401` tells linters that the “unused import” is intentional: nothing here uses the name `tricycle`, the import exists for its side effect. The `excavation` sub-package is *not* imported, which is the concrete reason the two tasks are independent (Section 1.3). When the excavation task is ready, a second line `from . import excavation # noqa: F401` registers it.

Watch out: a stray file that used to live at the repository root

Until this revision the repository also contained a file literally named `__init__ .py` (with a space) at the root, holding two import lines including `excavation`. A file with a space in its name cannot be imported, so it was dead; it has been deleted. The real package init is the one-liner above.

Chapter 8

tricycle/__init__.py: registering the task

Role: gives the task its public name and tells Gymnasium and Isaac Lab which three classes implement it. Runs once at start-up. 25 lines.

```
1  # Tricycle pipeline-validation task.
2  #
3  # Importing this module registers the task with Gymnasium. Isaac Lab imports
4  # it through the `isaacclab.tasks` entry point declared in pyproject.toml, so
5  # the CLI can find it:
6  #
7  # .\isaacclab.bat train --rl_library rsl_rl --task Luna-Tricycle-Direct --num_envs
   ↪ 4
8  # .\isaacclab.bat play --rl_library rsl_rl --task Luna-Tricycle-Direct --num_envs
   ↪ 1 --checkpoint latest --viz newton
9  #
10 # Success criterion: mean episode reward (= metres travelled in 2 s) climbs
11 # from ~0 and keeps climbing without the physics diverging.
```

Lines 1–11 Header comment. Lines 3–5 state the mechanism (entry point → import → registration); lines 7–8 are the two commands from the runbook, so anyone opening the file sees how to run it; lines 10–11 give the success criterion in the unit the reward is measured in (metres in 2 s).

```
12
13 import gymnasium as gym
14
15 from . import agents
16
17 gym.register(
18     id="Luna-Tricycle-Direct",
19     entry_point=f"{__name__}.tricycle_env:TricycleEnv",
20     disable_env_checker=True,
21     kwargs={
22         "env_cfg_entry_point": f"{__name__}.tricycle_env_cfg:TricycleEnvCfg",
23         "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:
   ↪ TricyclePPORunnerCfg",
24     },
25 )
```

Line 13 Gymnasium is the maintained fork of OpenAI Gym. Isaac Lab uses its registry as the phone book of tasks; nothing else from it is needed here.

Line 15 Imports the `agents` sub-package so that `agents.__name__` can be used on line 23 to spell the dotted path of the PPO config without hard-coding it. This also executes `agents/`

`init__.py`, which is empty.

Line 17 `gym.register` adds one entry to the global registry. Everything inside the call is stored as data; no class is imported yet.

Line 18 The task id. Isaac Lab develop's own tasks drop the `-v0` suffix the old scripts used, and so do we. `-task Luna-Tricycle-Direct` on the command line must match this string exactly.

Line 19 The environment class as a dotted path string, `module:ClassName`. `__name__` is `luna_hifi_tasks.tricycle`, so this reads `luna_hifi_tasks.tricycle.tricycle_env:TricycleEnv`. Gymnasium imports it lazily when `gym.make` is called (Section 3.5).

Line 20 Gymnasium normally wraps every environment in a checker that validates observation shapes against the declared space on each step. Isaac Lab environments return dictionaries of batched tensors, which the checker does not understand, so every Isaac Lab task disables it.

Line 21 Extra keyword arguments. Gymnasium passes them to the environment constructor, but Isaac Lab reads them earlier, from the registry, to find the config classes.

Line 22 `env_cfg_entry_point`: the environment config class. Isaac Lab's `load_cfg_from_registry(task, "env_cfg_entry_point")` imports `luna_hifi_tasks.tricycle.tricycle_env_cfg` and instantiates `TricycleEnvCfg` (Section 3.4).

Line 23 `rsl_rl_cfg_entry_point`: the agent config for the `rsl_rl` library, built from `agents.__name__` (`luna_hifi_tasks.tricycle.agents`) plus the module and class. The key name is a convention: `-rl_library rsl_rl` makes Isaac Lab look up `rsl_rl_cfg_entry_point`; a `skrl` backend would look for `skrl_cfg_entry_point`, and so on.

Line 24 Closes the `kwargs` dictionary.

Line 25 Closes the `gym.register` call.

Isaac Lab: the three classes behind one task id

Every Isaac Lab task is a triple: an *env class* (behaviour), an *env cfg class* (data: scene, physics, spaces), and one *agent cfg class per learning library*. The registry stores their import paths as strings so that listing tasks does not import heavy modules. The `gym.make` call and the two `load_cfg_from_registry` calls are where the strings are turned into classes.

Chapter 9

tricycle.py: the car model

Role: defines the car as an MJCF string and offers a helper to write it to disk for conversion. At run time only three constants are imported from it (CHASSIS_Z, JOINT_STEER, JOINT_REAR); the MJCF itself is used offline (Chapter 5). 156 lines.

9.1 Header

```
1  # tricycle.py
2  #
3  # The car, as an MJCF string. One steerable front wheel, two driven rear
4  # wheels, and a triangular frame connecting the three.
5  #
6  # This file is NOT read at training time. Training loads the converted USD in
7  # assets/tricycle/. The MJCF here is the *source* of that USD: whenever you
8  # change it, re-run the converter (see RUNBOOK.md, "Convert the tricycle
9  # asset") so the USD matches again.
10 #
11 # Body (plan view, +x forward):
12 #
13 #           rear_left           apex / steering head
14 #           o-----o           <- left rail
15 #           | [  deck  ] . '     front wheel hangs below the apex
16 #           o-----.'           <- right rail
17 #           rear_right
18 #           ^ rear crossbar
19 #
20 # Wheels are spheres, not cylinders: analytic spheres are the cheapest and most
21 # robust collider the MPM grid handles, and for "does the pipeline train" the
22 # contact patch shape is irrelevant. Wheel positions and radii are unchanged
23 # from the first working version, so nothing about the soil coupling moved.
24 #
25 # Every body with a joint has a geom with mass, so MuJoCo derives real inertia
26 # for all of them. A body with zero inertia produces NaNs in the solver.
27 #
28 # Frame: +x forward, +z up. Wheel bottoms sit at z = 0 in the chassis frame's
29 # rest pose (chassis centre at z = 0.14, wheel centres at z = 0.10, r = 0.10).
```

Lines 1–4 What the file is.

Lines 6–9 The most important operational fact about this file: it is *not* read during training. The USD under `assets/` is, and it must be regenerated when this file changes.

Lines 11–18 An ASCII plan view of the body so that the numbers below have a picture: the two rails meet at the apex over the front wheel; the crossbar closes the rear; the deck sits inside.

Lines 20–23 Why the wheels are spheres. The implicit MPM solver treats colliders as signed-distance fields; a sphere’s distance function is one subtraction and one square root, the cheapest and most robust collider it has. For “does the pipeline train” the shape of the contact patch does not matter. Line 22–23 records that wheel positions and radii were kept from the first working version so the soil coupling did not move.

Lines 25–26 A body whose geoms have zero total mass has a zero inertia tensor, and the solver divides by it. Every body here carries a geom with an explicit `mass`.

Lines 28–29 The coordinate convention: $+x$ forward, $+z$ up, wheel bottoms at $z = 0$ when the chassis origin is at $z = 0.14$.

9.2 Imports and dimensions

```

31 from __future__ import annotations
32
33 import math
34 import os
35 import tempfile
36
37 # -----
38 # Dimensions (metres, radians)
39 # -----
40
41 WHEEL_RADIUS = 0.10
42 CHASSIS_Z = 0.14          # chassis centre height with wheels resting on z = 0
43 WHEEL_Z = WHEEL_RADIUS - CHASSIS_Z  # wheel centre height relative to chassis
44     ↪ (-0.04)
45
46 FRONT_X = 0.22           # steering axis / front wheel centre
47 REAR_X = -0.15           # rear axle
48 REAR_Y = 0.16            # rear wheel centre, either side
49
50 # Triangle: apex at the steering head, base between the rear wheels.
51 APEX = (FRONT_X, 0.0)
52 REAR_CORNER_Y = 0.13     # inside the rear wheels, so the rails end inside them
53 RAIL_HALF_W = 0.02       # rail cross-section is 4 cm wide x 3 cm tall
54 RAIL_HALF_H = 0.015
55
56 # Masses: same 5.0 kg chassis total as the original box body.
57 RAIL_MASS = 0.8
58 CROSSBAR_MASS = 0.8
59 DECK_MASS = 2.6
60
61 JOINT_STEER = "steer"
62 JOINT_FRONT = "front_wheel"
63 JOINT_REAR = ["rear_left", "rear_right"]
64
65 FRAME_RGBA = "0.2 0.4 0.8 1"
66 DECK_RGBA = "0.55 0.6 0.7 1"
67 WHEEL_RGBA = "0.15 0.15 0.15 1"

```

Line 31 Lets the annotations on line 73 (`str | None, tuple[float, float]`) be written in the modern form on any Python 3.7+.

Lines 33–35 `math` for the rail geometry, `os` and `tempfile` for `write_mjcf`.

Line 41 Sphere radius of all three wheels, 10 cm.

Line 42 Height of the chassis body origin above the ground when the wheels rest on $z = 0$. `tricycle_env_cfg.py` imports this to compute the spawn height.

Line 43 Wheel centres relative to the chassis origin: $0.10 - 0.14 = -0.04$. Defined once so the three wheel bodies cannot drift apart.

Line 45 x of the steering axis and the front wheel centre, 22 cm ahead of the chassis origin.

Line 46 x of the rear axle, 15 cm behind. Wheelbase = 0.37 m.

Line 47 Half the rear track: rear wheel centres at $y = \pm 0.16$.

Line 50 The apex of the triangle, directly over the steering axis.

Line 51 The rear corners of the triangle sit at $y = \pm 0.13$, 3 cm inside the wheel centres, so the rails end inside the wheel spheres and the corners are visually capped by the wheels (Section 5.5).

Lines 52–53 Rail cross-section: half-width 2 cm, half-height 1.5 cm, i.e. 4 cm wide and 3 cm tall. MuJoCo box sizes are always half-extents.

Lines 56–58 Geom masses. $0.8 + 0.8 + 0.8 + 2.6 = 5.0$ kg, the same as the old single box, so the car's weight, wheel loads and drive torques are unchanged. Compiled with MuJoCo, the chassis centre of mass lands at $x = -0.054$ m (slightly behind the origin, towards the driven wheels) and the principal inertia is about $(0.062, 0.044, 0.019)$ kg m².

Lines 60–62 Joint names. These strings appear in the MJCF, survive the USD conversion as prim names, and are what `tricycle_env.py` passes to `find_joints`. `JOINT_REAR` is a list because the two rear wheels are always addressed together.

Lines 64–66 Colours as MJCF `rgba` strings. The converter copies them into `primvars:displayColor`, so they are what the Newton viewer shows.

9.3 Building the frame

```

68 # -----
69 # Geometry helpers
70 # -----
71
72
73 def _rail(name: str, start: tuple[float, float], end: tuple[float, float], mass:
    ↪ float) -> str:
74     """A box geom whose local x axis runs from `start` to `end` (chassis frame, z =
    ↪ 0)."""
75     dx, dy = end[0] - start[0], end[1] - start[1]
76     half_len = 0.5 * math.hypot(dx, dy)
77     yaw = math.atan2(dy, dx)
78     cx, cy = 0.5 * (start[0] + end[0]), 0.5 * (start[1] + end[1])
79     return (
80         f'<geom name="{name}" type="box" pos="{cx:.4f} {cy:.4f} 0" euler="0 0 {yaw}
    ↪ {:.6f}" '
81         f'size="{half_len:.4f} {RAIL_HALF_W} {RAIL_HALF_H}" mass="{mass}" rgba="{
    ↪ FRAME_RGBA}" />'
82     )
83
84
85 LEFT_RAIL = _rail("left_rail", (REAR_X, REAR_CORNER_Y), APEX, RAIL_MASS)

```

```

86 RIGHT_RAIL = _rail("right_rail", (REAR_X, -REAR_CORNER_Y), APEX, RAIL_MASS)
87
88 # Base of the triangle. Runs slightly past the rail ends so the corners are closed.
89 CROSSBAR = (
90     f'<geom name="crossbar" type="box" pos="{REAR_X} 0 0" '
91     f'size="{RAIL_HALF_W} {REAR_CORNER_Y + RAIL_HALF_W} {RAIL_HALF_H}" '
92     f'mass="{CROSSBAR_MASS}" rgba="{FRAME_RGBA}"/>'
93 )
94
95 # Flat floor pan inside the rear half of the triangle. Carries most of the mass.
96 DECK = (
97     f'<geom name="deck" type="box" pos="-0.08 0 0" size="0.06 0.065 0.01" '
98     f'mass="{DECK_MASS}" rgba="{DECK_RGBA}"/>'
99 )

```

Line 73 A helper that returns one `<geom>` element for a box whose local x axis runs from `start` to `end` in the chassis xy -plane. Writing the rails this way means the numbers are derived from the corner coordinates rather than typed by hand.

Line 74 Docstring.

Line 75 The vector from start to end.

Line 76 Half the rail length; MuJoCo wants half-extents. For the rails, $\frac{1}{2}\sqrt{0.37^2 + 0.13^2} = 0.1961$ m.

Line 77 The yaw angle of that vector. For the left rail (from $(-0.15, 0.13)$ to $(0.22, 0)$) it is $\text{atan2}(-0.13, 0.37) = -0.338$ rad $\approx -19.4^\circ$; the right rail gets $+0.338$.

Line 78 The rail's centre: the midpoint, $(0.035, \pm 0.065)$.

Lines 79–82 Emit the element. `pos` is the centre, `euler="0 0 yaw"` rotates about z (the compiler is told on line 107 that angles are radians), `size` is (half-length, half-width, half-height), `mass` fixes the mass (MuJoCo derives the density and the inertia), `rgba` the colour. The `name` lets the geom be found in the USD.

Line 85 The left rail: from the left rear corner to the apex.

Line 86 The right rail: mirror image.

Lines 89–93 The crossbar, a box centred on the rear axle line ($x = -0.15$), half-length in y of $0.13 + 0.02 = 0.15$ so that it covers the rails' ends, and the same cross-section as the rails.

Lines 96–99 The deck: a $12 \times 13 \times 2$ cm plate centred at $x = -0.08$, i.e. filling the rear half of the triangle where it is wide enough, 5 mm lower than the rails' top surface so it reads as a floor pan. It carries 2.6 of the 5 kg.

Why: why boxes and not a triangular mesh

MJCF has no triangle primitive. A mesh with inline vertices would survive MuJoCo's compiler, but boxes are the safest shape for every stage of the pipeline: the USD converter writes them as `Cube` prims with a scale, Newton's rigid collision pipeline handles boxes analytically, and the Isaac Lab visualiser draws them without any mesh import. Three rails plus a deck give an unmistakable triangle at zero risk.

9.4 The MJCF string

```

101 # -----
102 # Model
103 # -----
104
105 TRICYCLE_MJCF = f"""
106 <mujoco model="tricycle">
107   <compiler angle="radian" autolimits="true"/>
108   <default>
109     <geom condim="3" friction="0.8 0.005 0.0001"/>
110     <joint damping="0.02"/>
111   </default>
112
113   <worldbody>
114     <body name="chassis" pos="0 0 {CHASSIS_Z}">
115       <freejoint name="root"/>
116       {LEFT_RAIL}
117       {RIGHT_RAIL}
118       {CROSSBAR}
119       {DECK}
120
121       <!-- steering fork: hinge about z, carries the free-rolling front wheel -->
122       <body name="fork" pos="{FRONT_X} 0 {WHEEL_Z}">
123         <joint name="{JOINT_STEER}" type="hinge" axis="0 0 1" range="-0.6 0.6"/>
124         <geom type="capsule" fromto="0 0 0.02 0 0 -0.02" size="0.012" mass="0.1"/>
125         <body name="front_wheel_body" pos="0 0 0">
126           <joint name="{JOINT_FRONT}" type="hinge" axis="0 1 0"/>
127           <geom type="sphere" size="{WHEEL_RADIUS}" mass="0.5" rgba="{WHEEL_RGBA}"/>
128         </body>
129       </body>
130
131       <!-- driven rear wheels -->
132       <body name="rear_left_body" pos="{REAR_X} {REAR_Y} {WHEEL_Z}">
133         <joint name="{JOINT_REAR[0]}" type="hinge" axis="0 1 0"/>
134         <geom type="sphere" size="{WHEEL_RADIUS}" mass="0.5" rgba="{WHEEL_RGBA}"/>
135       </body>
136       <body name="rear_right_body" pos="{REAR_X} {-REAR_Y} {WHEEL_Z}">
137         <joint name="{JOINT_REAR[1]}" type="hinge" axis="0 1 0"/>
138         <geom type="sphere" size="{WHEEL_RADIUS}" mass="0.5" rgba="{WHEEL_RGBA}"/>
139       </body>
140     </body>
141   </worldbody>
142 </mujoco>
143 """

```

Line 105 An f-string: every `{...}` is filled in by Python at import time, so the constants above and the rail strings are pasted into the XML.

Line 106 Root element; `model` is just a label.

Line 107 Compiler options. `angle="radian"` makes `euler` and joint `range` values radians (the steering range on line 123 is ± 0.6 rad). `autolimits="true"` tells MuJoCo that a joint with a `range` is limited without needing a separate `limited="true"`.

Lines 108–111 Defaults inherited by every geom and joint. `condim="3"` gives contacts normal force plus two friction directions (sliding friction, no torsional/rolling terms). `friction="0.8 0.005 0.0001"` sets sliding, torsional and rolling friction coefficients; the converter copied these into the USD physics material (`physics:dynamicFriction = 0.8`, `newton:torsionalFriction`

`= 0.005, newton:rollingFriction = 0.0001)`. `joint damping="0.02"` adds a small viscous torque to every hinge so free wheels do not spin forever.

Line 113 Everything with a physical pose lives under `<worldbody>`.

Line 114 The chassis body, placed at $z = 0.14$ in the world when the model is loaded on its own.

Line 115 A free joint: six degrees of freedom, so the chassis floats and can translate and rotate freely. Isaac Lab treats a body with a free joint as a *floating-base* articulation, and its pose is the “root state” that `tricycle_env.py` reads and writes.

Lines 116–119 The four frame geoms, pasted in from lines 85–99.

Line 121 Comment.

Line 122 The fork body, a child of the chassis, at the front wheel centre. Its origin is the steering axis.

Line 123 The steering joint: a hinge about the fork’s z axis with range ± 0.6 rad ($\pm 34.4^\circ$). The USD converter writes this as `physics:lowerLimit = -34.377` degrees. `tricycle_env_cfg.py` commands it with a position target of at most ± 0.5 rad, inside the limit.

Line 124 A thin vertical capsule (radius 1.2 cm, from $z = +0.02$ to $z = -0.02$ in the fork frame) so the fork body has mass (0.1 kg) and hence inertia. It sits entirely inside the front wheel sphere.

Line 125 The front wheel body, a child of the fork, at the fork origin.

Line 126 The wheel’s spin axis: a hinge about y . No range, no actuator: the front wheel free-rolls on its default joint damping.

Line 127 The wheel sphere, 0.5 kg.

Lines 128–129 Close the wheel and fork bodies.

Line 131 Comment.

Lines 132–135 The left rear wheel: a body at $(-0.15, 0.16, -0.04)$ with a hinge about y named `rear_left` and a 0.5 kg sphere. This joint is velocity-driven from the env.

Lines 136–139 The right rear wheel, mirrored in y .

Lines 140–143 Close chassis, worldbody, mujoco, and the f-string.

Isaac Lab: what Isaac Lab needs from this model

Isaac Lab’s `Articulation` class needs: exactly one root body (the chassis, with its free joint), joints with stable names (for `find_joints`), and every body with mass. It gets the joint *order* from the USD; `find_joints` returns indices in that order, which is why the env never assumes a fixed index.

9.5 Writing the file

```
146 def write_mjcf(directory: str | None = None) -> str:
147     """Write the MJCF to disk and return its path. Idempotent.
148
149     Used only by the offline USD conversion step; nothing calls it at training time.
```

```
150     """
151     directory = directory or os.path.join(tempfile.gettempdir(), "luna_hifi_assets")
152     os.makedirs(directory, exist_ok=True)
153     path = os.path.join(directory, "tricycle.xml")
154     with open(path, "w") as fh:
155         fh.write(TRICYCLE_MJCF.strip() + "\n")
156     return path
```

Line 146 `write_mjcf` takes an optional directory and returns the path written.

Lines 147–150 Docstring: idempotent, offline use only. The runbook's conversion step calls this function through `python -c "from luna_hifi_tasks.tricycle.tricycle import write_mjcf; print(write_mjcf())"`.

Line 151 Default location: the system temp directory plus `luna_hifi_assets`. On Windows that is the folder the runbook's converter command reads from:

```
C:\Users\<you>\AppData\Local\Temp\luna_hifi_assets\tricycle.xml
```

Line 152 Create the folder if needed.

Line 153 The file is always called `tricycle.xml`.

Lines 154–155 Write the XML with the surrounding blank lines stripped and a final newline.

Line 156 Return the path so the caller can print it.

Why: no more write-at-import

The previous version of this file ended with `TRICYCLE_MJCF_PATH = write_mjcf()`, so every `import` of the package wrote a file into the temp directory, including on every training start, and the constant was never used. The line has been removed; the function is called explicitly when converting.

Chapter 10

tricycle_env_cfg.py: the environment configuration

Role: pure data. It describes the scene (ground, light, car, hidden slab, soil), the physics (Newton with a coupled rigid + MPM solver), the spaces, the time steps and the termination thresholds. It contains no behaviour except two methods on the `cfg` class: `__post_init__`, which assembles the physics config, and `play_mode`, which adjusts things for playback. 300 lines. It is the file where almost every hard-won fact from the runbook lives, so it gets the longest chapter.

Isaac Lab: configclass

`@configclass` (`source/isaacsim/isaacsim/utils/configclass.py`) is Isaac Lab's wrapper around Python's `dataclass`. It lets you write fields without type annotations, makes mutable defaults safe by deep-copying them per instance, adds `to_dict`, `from_dict`, `replace`, `copy` and `validate`, and calls `__post_init__` after the instance is built. A field set to `MISSING` must be filled in by a subclass or `validate()` raises. Everything in this file is a `configclass` instance; nothing is executed by the physics engine until `DirectRLEnv.__init__` reads it.

10.1 Header

```
1  # tricycle_env_cfg.py
2  #
3  # Barebones pipeline validation: a tricycle drives forward on a flat strip of
4  # MPM soil for 2 seconds. Reward is forward distance.
5  #
6  # Structure follows isaacsim_tasks/contrib/ur10_particle_push, which is the
7  # reference for running a rigid solver and implicit MPM together:
8  #   - NewtonCfg takes ONE solver_cfg. Rigid + MPM are two CouplerEntryCfg
9  #     entries inside a CouplerProxyCfg.
10 #   - The MPM entry must set in_place=True and all_particles=True.
11 #   - project_outside_colliders must be False on a coupled MPM entry.
12 #   - The tool bodies (here: wheels) are a CouplerProxyMappingCfg from the
13 #     rigid entry to the MPM entry, mode="lagged", with a mass_scale.
14 #   - physics is assigned in __post_init__, not as a class attribute.
15 #   - Scene prim_path uses {ENV_REGEX_NS}; coupler bodies use the expanded
16 #     /World/envs/env_.* form.
17 #
18 # The car is loaded from the USD in assets/tricycle/, which is converted
19 # offline from the MJCF in tricycle.py (see RUNBOOK.md). MJCF cannot be
20 # imported at runtime: the importer is a Kit extension and the CLI never
21 # boots Kit.
```

Lines 1–4 What the task is.

Lines 6–16 The design is copied from Isaac Lab’s own coupled task, `source/isaacsim_tasks/isaacsim_tasks/contrib/ur10_particle_push/ur10_particle_push_env_cfg.py`, which pushes particles with a robot paddle. The six bullets are the rules that task obeys and that this file obeys too; each reappears below next to the line that implements it.

Lines 18–21 Where the car comes from and why the MJCF cannot be loaded directly at run time (Chapter 5).

10.2 Imports

```

23 from __future__ import annotations
24
25 from pathlib import Path
26
27 import isaacsim.sim as sim_utils
28 from isaacsim.actuators import ImplicitActuatorCfg
29 from isaacsim.assets import ArticulationCfg, AssetBaseCfg, RigidBodyCfg
30 from isaacsim.envs import DirectREnvCfg
31 from isaacsim.scene import InteractiveSceneCfg
32 from isaacsim.sim import SimulationCfg
33 from isaacsim.sim.schemas import UsdPhysicsRigidBodyCfg
34 from isaacsim.sim.spawners.materials import RigidBodyMaterialBaseCfg
35 from isaacsim.utils.configclass import configclass
36
37 from isaacsim_newton.assets import MPMObjectCfg
38 from isaacsim_newton.physics import (
39     MJWarpSolverCfg,
40     MPMSolverCfg,
41     NewtonCfg,
42     NewtonCollisionPipelineCfg,
43 )
44 from isaacsim_newton.sim.schemas import NewtonCollisionPropertiesCfg
45 from isaacsim_newton.sim.spawners.mpm import MPMGridCfg, MPMParticleMaterialCfg
46
47 from isaacsim_contrib.coupling import CouplerEntryCfg, CouplerProxyCfg,
48     ↪ CouplerProxyMappingCfg
49
50 from .tricycle import CHASSIS_Z, JOINT_REAR, JOINT_STEER

```

Line 23 Postponed evaluation of annotations, so type hints can name classes freely.

Line 25 `pathlib.Path` for locating the asset relative to this file (line 83).

Line 27 `isaacsim.sim` is imported as `sim_utils` by Isaac Lab convention; it holds the spawner configs (`UsdFileCfg`, `CuboidCfg`, `GroundPlaneCfg`, `DomeLightCfg`).

Line 28 `ImplicitActuatorCfg`: a PD drive executed inside the physics solver (Chapter 4).

Line 29 The three asset config types used in the scene: an articulated robot, a plain rigid body, and a static scene element with no physics state of its own.

Line 30 The base class for direct-workflow environment configs.

Line 31 The base class for scene configs.

Line 32 The simulation config (time step, physics backend, visualisers).

Line 33 `UsdPhysicsRigidBodyCfg`: the schema config that marks a prim as a rigid body and can make it kinematic.

Line 34 `RigidBodyMaterialBaseCfg`: friction values for a rigid body.

Line 35 The `configclass` decorator. The runbook notes that this exact import path matters: an older from `isaacsim.utils` import `configclass` form, combined with stale `__pycache__`, once produced a confusing “`'module' object is not callable`”.

Line 37 `MPMObjectCfg`: the asset config for a particle body.

Lines 38–43 The Newton physics configs: the MuJoCo-Warp rigid solver, the MPM solver, the top-level `NewtonCfg`, and the collision-pipeline config.

Line 44 `NewtonCollisionPropertiesCfg`: per-prim Newton collision settings (margin, gap), used on the hidden slab.

Line 45 The particle spawner (`MPMGridCfg`) and its material (`MPMParticleMaterialCfg`).

Line 47 The coupler configs from the `isaacsim_contrib` package: an entry, the proxy coupler, and a proxy mapping (Section 4.4).

Line 49 From our own model file: the chassis height and the joint names. Nothing else from `tricycle.py` is needed at run time.

10.3 Constants

```

50 # -----
51 # Constants
52 # -----
53
54 RIGID_ENTRY = "rover"
55 MPM_ENTRY = "soil"
56
57 VOXEL_SIZE = 0.05
58 MPM_COLLIDER_MARGIN = 0.5 * VOXEL_SIZE
59 MPM_PARTICLES_PER_CELL = 1.0
60 MPM_VISUAL_COLOR = (0.62, 0.55, 0.45)
61
62 # Soil strip: 1.5 m long, 0.6 m wide, 6 cm deep. With a 5 cm voxel and one
63 # particle per cell that is 30 x 12 x 2 = 720 particles per env.
64 STRIP_LENGTH, STRIP_WIDTH, STRIP_DEPTH = 1.5, 0.6, 0.06
65 STRIP_START_X = -0.4                                # car spawns at x=0, on the strip
66
67 STRIP_LOWER = (STRIP_START_X, -0.5 * STRIP_WIDTH, MPM_COLLIDER_MARGIN)
68 STRIP_UPPER = (STRIP_START_X + STRIP_LENGTH, 0.5 * STRIP_WIDTH, MPM_COLLIDER_MARGIN
69                ↪ + STRIP_DEPTH)
69
70 # Hidden slab that gives the MPM entry a floor. Without it particles fall
71 # forever -- the MPM entry only sees bodies listed on its CouplerEntryCfg,
72 # not the global ground plane.
73 MPM_GROUND_SIZE = (STRIP_LENGTH + 1.0, STRIP_WIDTH + 1.0, 0.10)
74 MPM_GROUND_POSITION = (STRIP_START_X + 0.5 * STRIP_LENGTH, 0.0, -0.05)

```

Lines 54–55 The names of the two coupler entries. They are only used to link the proxy mapping to its source and destination (lines 272–273), but naming them once avoids typos.

Line 57 The MPM grid spacing, 5 cm. It sets soil resolution, the number of particles (line 62–63) and the collider margin below.

Line 58 Half a voxel. Used both as the collider margin on the slab (line 152) and as the vertical offset of the soil box (lines 67–68), so the lowest particle layer starts half a cell above the slab surface, where the implicit solver can see the collider before the particles touch it.

Line 59 One particle per grid cell along each axis. Higher values multiply the particle count by the cube of the factor.

Line 60 Soil colour (RGB, 0–1), used for the spawner and for the particle colour in the viewer.

Lines 62–65 The strip: 1.5 m long, 0.6 m wide, 6 cm deep, starting 40 cm behind the car so that the car spawns on it with room behind. With the resolution rule from Chapter 4, $[1.5/0.05] \times [0.6/0.05] \times [0.06/0.05] = 30 \times 12 \times 2 = 720$ particles per env.

Line 67 The lower corner of the particle box: rear end of the strip, left edge, half a voxel above $z = 0$.

Line 68 The upper corner: front end, right edge, strip depth above the lower z . Because `MPMGridCfg` coordinates are local to the asset prim and the asset's `init_state` is left at the origin (line 120), these are also env-local coordinates.

Lines 70–72 The rationale for the slab (also a green box in Chapter 4).

Line 73 Slab size: the strip plus a 1 m margin in x and y , 10 cm thick.

Line 74 Slab centre: under the middle of the strip, 5 cm below $z = 0$, so its top face lies exactly at $z = 0$.

10.4 The car

```

76 # -----
77 # Tricycle
78 # -----
79
80 # <repo>/assets/tricycle/tricycle.usda, found relative to this file so the
81 # repo can live anywhere. Regenerate it with the converter after editing
82 # tricycle.py (RUNBOOK.md, "Convert the tricycle asset").
83 ASSETS_DIR = Path(__file__).resolve().parents[2] / "assets"
84 TRICYCLE_USD_PATH = ASSETS_DIR / "tricycle" / "tricycle.usda"
85
86 TRICYCLE_CFG = ArticulationCfg(
87     prim_path="{ENV_REGEX_NS}/Tricycle",
88     spawn=sim_utils.UsdFileCfg(usd_path=str(TRICYCLE_USD_PATH)),
89     init_state=ArticulationCfg.InitialStateCfg(
90         pos=(0.0, 0.0, CHASSIS_Z + MPM_COLLIDER_MARGIN + STRIP_DEPTH + 0.01),
91         rot=(1.0, 0.0, 0.0, 0.0),
92         joint_pos={"*": 0.0},
93         joint_vel={"*": 0.0},
94     ),
95     actuators={
96         "steer": ImplicitActuatorCfg(
97             joint_names_expr=[JOINT_STEER],
98             effort_limit_sim=5.0,
99             velocity_limit_sim=5.0,
100             stiffness=20.0,
101             damping=1.0,
```

```

102     ),
103     "rear": ImplicitActuatorCfg(
104         joint_names_expr=JOINT_REAR,
105         effort_limit_sim=2.0,
106         velocity_limit_sim=30.0,
107         stiffness=0.0,           # velocity drive
108         damping=0.5,
109     ),
110     # front wheel free-rolls on MJCF joint damping -- no actuator
111 },
112 )

```

Lines 80–82 Comment: where the asset is and when to regenerate it.

Line 83 The repository root, found by walking up from this file: `parents[0]` is `tricycle/`, `parents[1]` is `luna_hifi_tasks/`, `parents[2]` is the repository. This replaced a hard-coded `C:\...` path, so the repository can be cloned anywhere.

Line 84 The USD entry file.

Line 86 An `ArticulationCfg` describes a robot: where to put it, what to spawn, its initial state, and its actuators.

Line 87 The prim path uses the `{ENV_REGEX_NS}` macro, which `InteractiveScene` expands to `/World/envs/env_.*`. One car is spawned under each env prim (Section 5.4).

Line 88 `UsdFileCfg` spawns a USD file by reference. `usd_path` must be a string, hence `str(...)`.

Line 89 Initial (and reset) state.

Line 90 Spawn position relative to the env origin: $x = y = 0$, and z equal to the chassis height that puts the wheel bottoms at $z = 0$, plus the soil offset, plus the soil depth, plus 1 cm clearance. The car therefore starts just above the soil surface and settles onto it in the first few steps.

Line 91 Orientation as a quaternion in Isaac Lab's (w, x, y, z) order; identity.

Lines 92–93 All joints start at zero position and zero velocity. The keys are regexes over joint names; `".*"` matches all.

Line 95 The actuator groups. Each key is a label; each value binds one actuator model to a set of joints selected by name.

Line 96 The steering group.

Line 97 Bind it to the joint named `steer`.

Line 98 Maximum torque the solver may apply, 5 N m.

Line 99 Maximum joint speed, 5 rad/s.

Line 100 PD stiffness 20 N m/rad: the drive pulls the fork towards the commanded angle.

Line 101 PD damping 1 N m s/rad.

Line 102 End of the steering group.

Line 103 The rear-wheel group, bound to both `rear_left` and `rear_right` (line 104).

Line 105 Torque limit 2 N m per wheel. With a 10 cm radius that is 20 N of tractive force per wheel, about 60% of the 6.6 kg car's weight in total, a plausible traction limit for soil.

Line 106 Speed limit 30 rad/s, above the 20 rad/s the policy can ask for.

Line 107 Stiffness zero: there is no position target, so the drive is a pure *velocity* controller.

Line 108 Damping 0.5 N ms/rad: the torque is $0.5 \times (\text{target speed} - \text{actual speed})$, saturating at the 2 N m limit.

Line 110 Comment: the front wheel has no actuator; it rolls freely with the MJCF's small joint damping.

Lines 111–112 Close the dictionary and the `cfg`.

Isaac Lab: how a PD implicit actuator becomes a velocity drive

An implicit drive computes $\text{torque} = \text{stiffness} \times (\text{pos target} - \text{pos}) + \text{damping} \times (\text{vel target} - \text{vel})$. With `stiffness=0` only the second term is left, so `set_joint_velocity_target` is all the env needs to call (Chapter 11). For the steering joint both gains are set and the env calls `set_joint_position_target`; its velocity target stays at the default of zero, so the damping term acts as a brake on fast steering.

10.5 The soil

```

114 # -----
115 # Soil
116 # -----
117
118 SOIL_CFG = MPMObjectCfg(
119     prim_path="{ENV_REGEX_NS}/Soil",
120     init_state=MPMObjectCfg.InitialStateCfg(),
121     spawn=MPMGridCfg(
122         lower=STRIP_LOWER,
123         upper=STRIP_UPPER,
124         voxel_size=VOXEL_SIZE,
125         particles_per_cell=MPM_PARTICLES_PER_CELL,
126         particle_placement="cell_center",
127         jitter=0.0,
128         # Packed ground, not a sandbox: high friction, capped compression.
129         material=MPMParticleMaterialCfg(
130             density=1800.0,
131             friction=0.84,           # tan(40 deg)
132             yield_pressure=1.0e12,
133         ),
134         visual_color=MPM_VISUAL_COLOR,
135     ),
136 )

```

Line 118 An `MPMObjectCfg` describes one particle body per env.

Line 119 Its prim under each env. Newton needs a prim to attach the particle cloud to.

Line 120 Default initial state: the particle box is not moved or rotated, so its local coordinates (lines 67–68) are env-local.

Line 121 The spawner: a regular lattice in a box.

Lines 122–123 The box corners.

Line 124 The target voxel size; together with line 125 it fixes the lattice resolution.

Line 125 One particle per cell along each axis.

Line 126 Put particles at cell centres, so each particle’s mass equals the cell volume times density and the total equals the box mass. The alternative, `"boundary"`, also places particles on the box faces.

Line 127 No random jitter: a perfectly regular lattice.

Line 128 Comment: the intent is firm ground, not loose sand.

Line 129 The material.

Line 130 Density 1800 kg/m^3 , a dense regolith or packed soil. Each of the 720 particles weighs $0.05^3 \times 1800 = 0.225 \text{ kg}$; a strip is 162 kg.

Line 131 Friction coefficient $0.84 = \tan 40^\circ$: a steep internal friction angle. With zero `yield_stress` (cohesion) the material is a pure frictional granular medium.

Line 132 Yield pressure 10^{12} Pa : the soil never yields in compression, so wheels cannot compact it away. The defaults of the other fields (`young_modulus` 10^{15} Pa , `poisson_ratio` 0.3, no viscosity, damping, hardening or dilatancy) are inherited.

Line 133 End of material.

Line 134 Colour handed to the particle visualiser.

Lines 135–136 Close spawner and `cfg`.

Watch out: parameter semantics changed from the old custom MPM

In Newton’s implicit MPM, `friction` is $\tan \phi$ and `yield_stress` is cohesion in pascals; the yield surface is $\tau_{\max}(p) = \text{yield_stress} + \text{friction} \cdot (p - p_{\min})$. Numbers from a Drucker–Prager α or a log-strain cohesion model do not carry over; the runbook says “recalibrate, don’t port”. There are also no `hardening_rate`/`softening_rate` fields.

10.6 The hidden slab

```

139 def _mpm_ground() -> RigidBodyCfg:
140     """Hidden kinematic slab so soil has a floor inside the MPM entry."""
141     return RigidBodyCfg(
142         prim_path="{ENV_REGEX_NS}/MPMGround",
143         init_state=RigidBodyCfg.InitialStateCfg(pos=MPM_GROUND_POSITION),
144         spawn=sim_utils.CuboidCfg(
145             size=MPM_GROUND_SIZE,
146             rigid_props=UsdPhysicsRigidBodyCfg(
147                 rigid_body_enabled=True,
148                 kinematic_enabled=True,
149             ),
150             collision_props=NewtonCollisionPropertiesCfg(
151                 collision_enabled=True,
152                 contact_margin=MPM_COLLIDER_MARGIN,
153                 contact_gap=0.0,
154             ),

```

```

155         physics_material=RigidBodyMaterialBaseCfg(
156             static_friction=0.8,
157             dynamic_friction=0.7,
158         ),
159         visible=False,
160     ),
161 )

```

Line 139 A small factory function returning the slab config. A function rather than a constant keeps the long expression readable and mirrors the reference task.

Line 140 Docstring: the whole reason for the slab.

Line 141 A `RigidBodyCfg`: a single rigid body per env with no joints.

Line 142 Its prim, `MPMGround` under each env. This exact name is what the MPM coupler entry lists as its body (line 262).

Line 143 Placed at the slab centre from line 74.

Line 144 Spawn a box primitive.

Line 145 With the size from line 73.

Lines 146–149 Rigid-body schema: enabled, and *kinematic*. A kinematic body has a pose but no dynamics: it does not fall, and nothing can push it. It exists only to be a collider.

Lines 150–154 Newton collision schema: collisions on; a `contact_margin` of half a voxel inflates the slab's collision surface outward by 2.5 cm so the MPM grid sees it early; `contact_gap` zero adds no extra detection distance.

Lines 155–158 Friction of the slab against particles: static 0.8, dynamic 0.7.

Line 159 Invisible: it is a physics-only helper and would otherwise hide the soil in the viewer.

Lines 160–161 Close.

10.7 The scene

```

164 # -----
165 # Scene
166 # -----
167
168
169 @configclass
170 class TricycleSceneCfg(InteractiveSceneCfg):
171     ground = AssetBaseCfg(
172         prim_path="/World/GroundPlane",
173         spawn=sim_utils.GroundPlaneCfg(),
174         collision_group=-1,
175     )
176     light = AssetBaseCfg(
177         prim_path="/World/Light",
178         spawn=sim_utils.DomeLightCfg(color=(0.8, 0.8, 0.8), intensity=2500.0),
179     )
180
181     tricycle: ArticulationCfg = TRICYCLE_CFG
182     mpm_ground: RigidBodyCfg = _mpm_ground()

```

```
183     soil: MPMObjectCfg = SOIL_CFG
```

Line 169 A configclass ...

Line 170 ... inheriting `InteractiveSceneCfg`, which contributes `num_envs`, `env_spacing`, `replicate_physics` and `friends`. Every attribute added here becomes an entry of the scene.

Lines 171–175 The ground plane. `AssetBaseCfg` is for things that have a prim but no per-env state; its path is absolute (one plane for all envs), spawned by `GroundPlaneCfg` (a 100×100 m plane with a default material). `collision_group=-1` puts it in the global collision group so it collides with every env's bodies instead of only with one env. It belongs to the rigid coupler entry via `include_static_shapes=True` (line 236); the soil never sees it.

Lines 176–179 A dome light, so the OpenGL viewer is not dark. Colour and intensity are cosmetic.

Line 181 The car: the scene entry named `tricycle`, which is what `scene["tricycle"]` returns in the env.

Line 182 The slab, entry name `mpm_ground`.

Line 183 The soil, entry name `soil`.

Isaac Lab: scene entries and their order

`InteractiveScene` iterates the cfg's fields in declaration order and spawns each. Order barely matters here, but it decides the order in which Newton is handed bodies, and the entry *names* are the keys used by `scene[...]` and by the reset (`scene.reset(env_ids)` calls `reset` on every asset).

10.8 The environment config

```
186 # -----
187 # Environment
188 # -----
189
190
191 @configclass
192 class TricycleEnvCfg(DirectRLEnvCfg):
193     decimation = 2                # policy at 50 Hz on a 100 Hz sim
194     episode_length_s = 2.0        # "how far in 2 seconds"
195
196     sim: SimulationCfg = SimulationCfg(dt=1.0 / 100.0, render_interval=decimation)
197
198     scene: TricycleSceneCfg = TricycleSceneCfg(
199         num_envs=8,
200         env_spacing=2.0,
201         replicate_physics=True,
202     )
203
204     # actions: [throttle, steer] in [-1, 1]
205     action_space = 2
206     max_wheel_speed = 20.0        # rad/s at throttle = 1 (~2 m/s at r = 0.1)
207     max_steer = 0.5               # rad at steer = +-1
208
209     # obs: lin vel b (3) + ang vel b (3) + proj gravity (3)
210     #       + steer pos (1) + steer vel (1) + rear wheel vel (2) + last action (2)
211     observation_space = 15
```

```

212     state_space = 0
213
214     # termination
215     max_lateral_drift = 0.5           # m; off the strip sideways
216     flip_gravity_z = 0.3           # projected gravity z above this => flipped
217
218     # Scales the wheel inertia MPM sees. Stability knob for the lagged
219     # feedback -- lower it if the car starts chattering on the strip.
220     proxy_mass_scale: float = 10.0

```

Line 191 A configclass ...

Line 192 ... inheriting `DirectRLEnvCfg`, which declares the required fields (`decimation`, `episode_length_s`, `scene`, `observation_space`, `action_space`) and defaults for the rest (`sim`, `seed`, noise models, `events`, ...).

Line 193 Two physics steps per policy step. With `dt = 1/100` the policy acts at 50 Hz.

Line 194 Episodes last 2 s. `DirectRLEnv` converts this to `max_episode_length = ⌈2.0/(0.01 × 2)⌉ = 100` control steps.

Line 196 The simulation config: 10 ms physics step and a render every `decimation` physics steps, i.e. once per control step (rendering more often than the policy acts would be wasted work; rendering less often triggers a warning). The `physics` field is filled in later by `__post_init__`.

Lines 198–202 The scene from above with its replication settings: 8 envs by default (the runbook trains with `-num_envs 4`), 2 m apart on the env grid, and `replicate_physics=True` (spawn env 0 and clone it rather than spawning every env from scratch, which is much faster and is what Newton’s per-world builder expects).

Line 204 Comment.

Line 205 Two actions. An `int` becomes `Box(-inf, inf, (2,))`; the env clamps to `[-1, 1]` itself.

Line 206 Action scale for throttle: at +1 the rear wheels are asked to spin at 20 rad/s, about 2 m/s at 10 cm radius. (The runbook’s “known-good” table lists 8.0 for this knob; the committed value is 20.0. Both have been run; lower values make the car slower and gentler on the coupling.)

Line 207 Action scale for steering: ± 0.5 rad, inside the joint’s ± 0.6 rad limit.

Lines 209–210 Comment listing the observation vector; the sum is 15.

Line 211 The observation size. Becomes `Box(-inf, inf, (15,))` under the “policy” key.

Line 212 No separate critic state; 0 means “no state space” (Isaac Lab treats a falsy value as absent). The critic sees the same 15 numbers.

Line 214 Comment.

Line 215 If the car drifts more than 0.5 m sideways it has left the 0.6 m wide strip (with a little slack because the wheels are 32 cm apart).

Line 216 Flip threshold on the z component of gravity expressed in the car’s body frame; see the orange box below.

Lines 218–220 A named knob for the proxy mass scale so that it can be swept from the command line (`env.proxy_mass_scale=5`) without editing the file. Line 276 reads it.

Watch out: projected gravity reads +1 upright for this asset

`projected_gravity_b` is the world gravity direction rotated into the body frame. For most Isaac Lab robots “upright” gives $(0, 0, -1)$. For this converted asset it gives $(0, 0, +1)$, so the flip test in `tricycle_env.py` is `projected_gravity_b[:, 2] < 0.3`: below 0.3 means the car has rolled more than about 73° from upright. If you ever swap the asset, print this quantity once before trusting the termination.

10.9 `__post_init__`: the physics

```

223     def __post_init__(self) -> None:
224         self.sim.physics = NewtonCfg(
225             solver_cfg=CouplerProxyCfg(
226                 entries=[
227                     CouplerEntryCfg(
228                         name=RIGID_ENTRY,
229                         solver_cfg=MJWarpSolverCfg(
230                             use_mujoco_contacts=False,
231                             integrator="implicitfast",
232                             njmax=128,
233                             nconmax=128,
234                         ),
235                         bodies=[r"/World/envs/env_*/Tricycle"],
236                         include_static_shapes=True,
237                         substeps=2,
238                     ),
239                     CouplerEntryCfg(
240                         name=MPM_ENTRY,
241                         solver_cfg=MPMSolverCfg(
242                             voxel_size=VOXEL_SIZE,
243                             grid_type="sparse",
244                             grid_padding=0,
245                             strain_basis="P0",
246                             transfer_scheme="apic",
247                             max_iterations=24,
248                             tolerance=1.0e-4,
249                             warmstart_mode="auto",
250                             velocity_basis="Q1",
251                             collider_basis="pic27",
252                             collider_velocity_mode="forward",
253                             solver="auto",
254                             separate_worlds=True,
255                             # Must be False on a coupled entry.
256                             project_outside_colliders=False,
257                             max_active_cell_count=1 << 14,
258                             max_leaf_node_count=1 << 13,
259                             max_lower_node_count=1 << 12,
260                             max_upper_node_count=1 << 10,
261                         ),
262                         bodies=[r"/World/envs/env_*/MPMGround"],
263                         all_particles=True,
264                         include_static_shapes=False,
265                         include_child_joints=False,
266                         substeps=1,
267                         in_place=True,

```

```

268         ),
269     ],
270     proxies=[
271         CouplerProxyMappingCfg(
272             source=RIGID_ENTRY,
273             destination=MPM_ENTRY,
274             bodies=[r"/World/envs/env_*/Tricycle/Geometry/chassis/.*_
    ↪ body"],
275             mode="lagged",
276             mass_scale=self.proxy_mass_scale,
277             collision_pipeline=None,
278         )
279     ],
280     iterations=1,
281 ),
282 collision_cfg=NewtonCollisionPipelineCfg(soft_contact_max=0),
283 num_substeps=1,
284 use_cuda_graph=True,
285 )
286 self.sim.use_newton_actuators = True

```

Line 223 `configclass` calls this after the instance is built. Setting the physics here rather than as a class attribute is deliberate: it can read `self.proxy_mass_scale`, and it guarantees a fresh `NewtonCfg` per instance.

Line 224 Assign a Newton backend to `sim.physics`. This single assignment is what makes the whole run use Newton instead of PhysX.

Line 225 The solver is a *coupler* holding two entries and one proxy mapping (Figure 4.2).

Line 226 The list of entries.

Lines 227–228 Entry one, named `rover`.

Lines 229–234 Its solver: MuJoCo-Warp with Newton-side contacts, the `implicitfast` integrator, and 128 constraint rows and 128 contacts per env (Chapter 4 explains each).

Line 235 The bodies this entry owns: every body under each env’s `Tricycle` prim (descendants are included by the regex semantics), i.e. the whole car.

Line 236 Also own every shape that has no body: the ground plane. Without this the car would fall through the world floor.

Line 237 Two rigid sub-steps of 5 ms inside each 10 ms coupled step, for accuracy of the stiff wheel drives.

Line 238 End of entry one.

Lines 239–240 Entry two, named `soil`.

Line 241 An implicit MPM solver.

Lines 242–254 Grid and numerics, all explained in Section 4.3: 5 cm sparse grid without padding, P0 strain, APIC transfer, at most 24 iterations to 10^{-4} , automatic warm start and solver choice, Q1 velocities, 27-point collider sampling with forward velocities, and one independent grid per env.

Lines 255–256 Rule from the header: no particle push-out on a coupled entry.

Lines 257–260 Sparse-grid capacities shared by all envs, in the required order $\text{active} \geq \text{leaf} \geq \text{lower} \geq \text{upper}$: 16384/8192/4096/1024. These are the 4-env, 6 GB numbers. For more envs raise all four roughly in proportion; `-num_envs` does not do it for you (Section 3.4).

Line 261 End of the MPM solver config.

Line 262 The only body the soil solver knows: the slab. Everything the soil should collide with must either be here or arrive as a proxy.

Line 263 This entry owns all particles in the model.

Line 264 It does not own the body-less shapes (the ground plane); those went to the rigid entry.

Line 265 It owns no joints (the slab has none).

Line 266 One MPM sub-step per coupled step.

Line 267 Step in place, mandatory for MPM.

Lines 268–269 Close entry two and the list.

Line 270 The proxy mappings.

Line 271 One mapping.

Lines 272–273 From the rigid entry into the soil entry.

Line 274 Which bodies to mirror: the three wheel bodies. The regex matches `rear_left_body`, `rear_right_body` and `fork/front_wheel_body` because `.*` also matches the slash; it does not match `chassis` or `fork`, which never touch soil.

Line 275 Lagged mode: poses go over at the start of the step, impulses come back one step late (Section 4.4).

Line 276 The proxies weigh ten wheels each ($0.5 \text{ kg} \times 10$), from the knob on line 220.

Line 277 `None` means the soil entry reuses the shared outer contacts instead of running its own collision pipeline for the proxies.

Lines 278–279 Close the mapping and the list.

Line 280 One relaxation pass per coupled step.

Line 281 End of the coupler.

Line 282 Newton’s rigid collision pipeline, with the particle-vs-shape (“soft”) contact buffer disabled because the MPM solver handles particle contacts itself.

Line 283 One coupled solve per physics step.

Line 284 Capture the step as a CUDA graph.

Line 285 End of `NewtonCfg`.

Line 286 Let Newton execute the actuator drives natively.

10.10 `play_mode`: settings for playback

```
288     def play_mode(self) -> None:
289         from isaacsim.visualizers.newton import NewtonGLVisualizerCfg
290
291         super().play_mode()
292         self.scene.num_envs = min(self.scene.num_envs, 4)
293         self.sim.visualizer_cfgs = [
294             NewtonGLVisualizerCfg(
295                 show_particles=True,
296                 particle_color=MPM_VISUAL_COLOR,
297                 eye=(2.0, 2.0, 1.5),
298                 lookat=(0.5, 0.0, 0.0),
299             )
300         ]
```

Line 288 Isaac Lab calls this method on the env cfg when running `play` (Section 3.9). It never runs during training.

Line 289 The visualiser config is imported inside the method so that a headless training machine without the visualiser package can still import this file.

Line 291 The base implementation caps `num_envs` at 50 and switches off observation noise.

Line 292 Cap further to 4 envs: the soil grid capacities were sized for 4.

Line 293 Attach one visualiser to the simulation.

Line 294 Newton’s OpenGL viewer.

Line 295 Draw the MPM particles. The default is `False`, which is why “soil invisible” is a runbook troubleshooting entry.

Line 296 Draw them in the soil colour.

Lines 297–298 Camera position and target, in metres, looking at the middle of the strip from front-left and above.

Lines 299–300 Close.

Chapter 11

tricycle_env.py: the environment

Role: the behaviour. It fills in the six hooks that `DirectREnv` leaves abstract (`_setup_scene`, `_pre_physics_step`, `_apply_action`, `_get_observations`, `_get_rewards`, `_get_dones`) plus the reset hook `_reset_idx`. Every method operates on tensors with a leading env dimension. 120 lines. Read Section 3.8 first: the order in which `DirectREnv.step` calls these methods is what makes them correct.

11.1 Header, imports, constructor

```
1  # tricycle_env.py
2  #
3  # The smallest DirectREnv that exercises the whole stack: MJCF articulation,
4  # MPM soil, two-way contact, batched reset, rsl_rl. Reward is per-step forward
5  # displacement, which sums to total distance over the 2 s episode.
6
7  from __future__ import annotations
8
9  import torch
10
11  from isaacsim.assets import Articulation
12  from isaacsim.envs import DirectREnv
13
14  from isaacsim_newton.assets import MPMObject
15
16  from .tricycle import JOINT_REAR, JOINT_STEER
17  from .tricycle_env_cfg import TricycleEnvCfg
18
19
20  class TricycleEnv(DirectREnv):
21      cfg: TricycleEnvCfg
22
23      def __init__(self, cfg: TricycleEnvCfg, render_mode: str | None = None, **kwargs
24      ↪ ):
25          super().__init__(cfg, render_mode, **kwargs)
26
27          self._steer_id, _ = self.car.find_joints([JOINT_STEER])
28          self._rear_ids, _ = self.car.find_joints(JOINT_REAR)
29          assert len(self._steer_id) == 1 and len(self._rear_ids) == 2
30
31          self._actions = torch.zeros(self.num_envs, 2, device=self.device)
32          self._prev_x = torch.zeros(self.num_envs, device=self.device)
```

Lines 1–5 What the file exercises. “Reward is per-step forward displacement, which sums to total distance” is the key design fact: the episode return equals metres driven.

Line 7 Modern annotation syntax.

Line 9 PyTorch; every quantity is a torch tensor on the GPU.

Line 11 `Articulation`, used only as a type annotation on line 38.

Line 12 The base class.

Line 14 `MPMObject`, used as a type annotation on line 39.

Line 16 Joint names from the model file.

Line 17 The `cfg` class, for the `cfg`: annotation and for the constructor signature.

Line 20 The `env` class. `DirectRLEnv` is also a `gym.Env`, so `gym.make` can build it.

Line 21 A class-level annotation telling type checkers (and readers) that `self.cfg` is a `TricycleEnvCfg`, so `self.cfg.max_steer` is known to exist.

Line 23 The constructor signature Isaac Lab expects: the `cfg`, an optional render mode, and pass-through keyword arguments.

Line 24 Runs everything in Section 3.5: simulation context, scene, our `_setup_scene`, physics start, buffers. When it returns, `self.car` exists and its data is live.

Line 26 Look up the steering joint's index by name. `find_joints` returns (`indices`, `names`); we keep the indices. The result is a list with one element.

Line 27 Likewise the two rear-wheel indices, in the order given (`rear_left`, `rear_right`).

Line 28 Fail loudly at start-up if the asset does not contain exactly those joints, rather than producing wrong tensors later. This assertion is what would fire if the USD were regenerated with renamed joints.

Line 30 A buffer for the last action, shape $(N, 2)$, on the simulation device. It is reused every step to avoid allocations and is also part of the observation.

Line 31 A buffer for each car's x position at the previous control step, shape $(N,)$, used by the reward.

Isaac Lab: `self.num_envs` and `self.device`

Both come from `DirectRLEnv`: `num_envs` is `cfg.scene.num_envs` after command-line overrides, and `device` is `cfg.sim.device` (`cuda:0`). All buffers you create must live on `self.device` or every step pays a host-device copy.

11.2 Scene hook

```

33     # ----- scene
34
35     def _setup_scene(self):
36         # InteractiveScene has already spawned everything in TricycleSceneCfg.
37         # Constructing assets here would try to spawn them a second time.
38         self.car: Articulation = self.scene["tricycle"]
39         self.soil: MPMObject = self.scene["soil"]

```

Line 35 Called by `DirectRLEnv.__init__` after `InteractiveScene(self.cfg.scene)` has spawned everything and before the simulation starts.

Lines 36–37 The trap this file used to fall into: constructing an `Articulation(cfg)` here would try to spawn the prim a second time and fail with “A prim already exists at path”. Isaac Lab’s tutorial tasks create assets here because their scene cfgs do not list them; ours does.

Line 38 Fetch the car by its scene entry name (line 181 of the cfg). The annotation makes later attribute access readable in an editor.

Line 39 Fetch the soil. It is only needed for `reset`.

11.3 Actions

```

41     # ----- actions
42
43     def _pre_physics_step(self, actions: torch.Tensor):
44         self._actions[:] = actions.clamp(-1.0, 1.0)
45
46     def _apply_action(self):
47         throttle = self._actions[:, 0:1] * self.cfg.max_wheel_speed
48         steer = self._actions[:, 1:2] * self.cfg.max_steer
49         self.car.set_joint_velocity_target(throttle.expand(-1, 2), joint_ids=self._
    ↪ rear_ids)
50         self.car.set_joint_position_target(steer, joint_ids=self._steer_id)

```

Line 43 Called once per control step with the raw policy output, shape $(N, 2)$, before any physics.

Line 44 Clamp to $[-1, 1]$ and store *in place* into the buffer (`[:]`). The policy is a Gaussian and can emit values outside the range; clamping keeps the scales on the next lines meaningful. In-place assignment keeps the same tensor object that `_get_observations` concatenates.

Line 46 Called once per *physics* sub-step (twice per control step with `decimation=2`), or once per control step when Newton folds the decimation loop into one call. Either way it writes the same targets.

Line 47 Throttle in rad/s: column 0 (kept as an $(N, 1)$ slice with `0:1`) times `max_wheel_speed`.

Line 48 Steering in rad: column 1 times `max_steer`.

Line 49 Write the same speed to both rear wheels. `expand(-1, 2)` turns $(N, 1)$ into a view of shape $(N, 2)$ without copying; `joint_ids` selects the two rear joints found in the constructor. Because the rear actuators have zero stiffness, this target alone defines the drive torque.

Line 50 Write the steering angle as a position target to the steering joint. Targets are buffered by the `Articulation` and flushed to the solver by `scene.write_data_to_sim()` right after this method (Figure 3.2).

Why: no differential steering

Both rear wheels get the same speed target and only the front wheel steers. The car therefore cannot “skid-steer” by spinning the rear wheels at different speeds; every turn comes from the fork. It keeps the action space at two numbers and makes the steering observation meaningful.

11.4 Observations

```

52     # ----- observations
53
54     def _get_observations(self) -> dict:
55         d = self.car.data
56         obs = torch.cat(
57             [
58                 d.root_lin_vel_b,
59                 d.root_ang_vel_b,
60                 d.projected_gravity_b,
61                 d.joint_pos[:, self._steer_id],
62                 d.joint_vel[:, self._steer_id],
63                 d.joint_vel[:, self._rear_ids],
64                 self._actions,
65             ],
66             dim=-1,
67         )
68         return {"policy": obs}

```

Line 54 Called at the end of every step, after resets. Must return a dict with at least a "policy" entry whose tensor matches `observation_space`.

Line 55 A short alias for the car's data container. Its fields are recomputed by `scene.update` each sub-step.

Line 56 Concatenate along the last dimension ...

Line 58 ... the root linear velocity in the body frame, $(N, 3)$: forward speed, sideways speed, vertical speed as the car feels them;

Line 59 the root angular velocity in the body frame, $(N, 3)$: roll, pitch, yaw rates;

Line 60 gravity projected into the body frame, $(N, 3)$: a compact encoding of roll and pitch that does not need a heading (yaw is deliberately not observed, so the policy is heading-independent);

Line 61 the steering angle, $(N, 1)$;

Line 62 the steering rate, $(N, 1)$;

Line 63 the two rear-wheel speeds, $(N, 2)$, which together with forward speed let the policy infer slip;

Line 64 the previous action, $(N, 2)$, which helps the policy produce smooth commands;

Lines 65–67 ... for a total of $3 + 3 + 3 + 1 + 1 + 2 + 2 = 15$, matching the `cfg`.

Line 68 Return under the "policy" key. The `rs1_rl` wrapper feeds the same group to actor and critic because no `obs_groups` are configured.

Why: no position in the observation

The car's x position is not observed. The reward is displacement, not position, so the optimal action does not depend on where along the strip the car is; leaving position out removes a feature the policy could only misuse and keeps the observation translation-invariant.

11.5 Reward

```

70 # ----- rewards
71
72 def _get_rewards(self) -> torch.Tensor:
73     x = self.car.data.root_pos_w[:, 0] - self.scene.env_origins[:, 0]
74     dx = torch.nan_to_num(x - self._prev_x, nan=0.0, posinf=0.0, neginf=0.0)
75     self._prev_x[:] = x
76     return dx.clamp(-0.1, 0.1)      # 0.1 m per 20 ms step = 5 m/s ceiling

```

Line 72 Called once per step, after `_get_dones` and before resets. Returns $(N,)$.

Line 73 Each car’s world x minus its env origin’s x : the env-local forward position, so cars in different envs are comparable.

Line 74 The displacement since last step, with any NaN or infinity replaced by zero. If the physics diverged this step, the reward is zero rather than garbage, and `_get_dones` will terminate the env.

Line 75 Remember the position for next step (in place).

Line 76 Clamp to ± 0.1 m per 20 ms step, i.e. ± 5 m/s. A car driving at its top speed of about 2 m/s earns 0.04 per step; the clamp only bites on non-physical jumps, so a diverging simulation cannot pay out a giant reward before it is terminated. Summed over an episode this is metres travelled forward.

11.6 Terminations

```

78 # ----- dones
79
80 def _get_dones(self) -> tuple[torch.Tensor, torch.Tensor]:
81     time_out = self.episode_length_buf >= self.max_episode_length - 1
82
83     pos = self.car.data.root_pos_w - self.scene.env_origins
84     vel = self.car.data.root_lin_vel_b
85
86     invalid = ~torch.isfinite(pos).all(dim=-1) | ~torch.isfinite(vel).all(dim
    ↳ =-1)
87     invalid |= pos.abs().max(dim=-1).values > 50.0
88     invalid |= vel.abs().max(dim=-1).values > 50.0
89
90     off_strip = pos[:, 1].abs() > self.cfg.max_lateral_drift
91     flipped = self.car.data.projected_gravity_b[:, 2] < self.cfg.flip_gravity_z
92
93     return invalid | off_strip | flipped, time_out

```

Line 80 Returns two boolean $(N,)$ tensors: `terminated` and `time_out` (Section 2.6).

Line 81 Time-out when the per-env step counter reaches `max_episode_length - 1`, i.e. after 100 control steps. This is the idiom every Isaac Lab direct task uses (the counter was already incremented for this step).

Line 83 Env-local position of each car, $(N, 3)$.

Line 84 Body-frame linear velocity, $(N, 3)$.

Line 86 “Invalid” if any component of position or velocity is NaN or infinite: the physics has blown up.

Line 87 ... or the car is more than 50 m from its origin;

Line 88 ... or moving faster than 50 m/s. Both are impossible for a real car and catch a diverging solver a step or two before it produces NaNs.

Line 90 Off the strip sideways: $|y|$ beyond `max_lateral_drift`.

Line 91 Flipped: body-frame gravity z below the threshold (orange box in Chapter 10).

Line 93 Terminate on any of the three; time out separately. An env can be both terminated and timed out in the same step; `rsl_rl` treats `terminated` as taking priority.

Why: why the invalid-state termination exists

The runbook records a run where the reward exploded to 10^8 : a coupled step had diverged, the car was flung away at absurd speed, and the displacement reward faithfully paid for it. Terminating on non-finite or absurd state, together with the reward clamp, turns a diverging env into a short, low-reward episode instead of a poisoned training batch.

11.7 Reset

```

95     # ----- reset
96
97     def _reset_idx(self, env_ids: torch.Tensor | None):
98         if env_ids is None:
99             env_ids = torch.arange(self.num_envs, device=self.device)
100         super()._reset_idx(env_ids)
101
102         root_state = self.car.data.default_root_state[env_ids].clone()
103         root_state[:, :3] += self.scene.env_origins[env_ids]
104         self.car.write_root_pose_to_sim(root_state[:, :7], env_ids)
105         self.car.write_root_velocity_to_sim(root_state[:, 7:], env_ids)
106         self.car.write_joint_state_to_sim(
107             self.car.data.default_joint_pos[env_ids],
108             self.car.data.default_joint_vel[env_ids],
109             None,
110             env_ids,
111         )
112
113         # Puts every particle of these envs back at its spawn position with zero
114         # velocity. Solver-side history (warm starts, plastic strain) is not
115         # cleared by this call; NewtonMPMManager.reset_solver_state exists for
116         # that, and the tricycle trains fine without it.
117         self.soil.reset(env_ids)
118
119         self._actions[env_ids] = 0.0
120         self._prev_x[env_ids] = root_state[:, 0] - self.scene.env_origins[env_ids],
        ↪ 0]

```

Line 97 Called with the indices of the envs that ended this step, and once with all indices at start-up (from `DirectRLEnv.reset`).

Lines 98–99 Defensive: if ever called with `None`, reset everything. `DirectRLEnv` never passes `None`,

so this is a safety net (the previous version referenced an attribute that does not exist on `Articulation`; it has been replaced with `torch.arange`).

Line 100 The base implementation: `scene.reset(env_ids)` (which calls `reset` on every asset and clears their internal buffers), applies any reset-mode events, and zeroes `episode_length_buf[env_ids]`.

Line 102 Start from the default root state of the selected envs (position, quaternion, linear and angular velocity: 13 numbers per env, from the `cfg's init_state`). `clone()` because the next line modifies it.

Line 103 Add each env's origin to the position, turning env-local spawn coordinates into world coordinates.

Line 104 Write pose (7 numbers: position and quaternion) to the simulator for those envs.

Line 105 Write velocity (6 numbers) for those envs: all zero.

Lines 106–111 Write joint positions and velocities back to their defaults (all zero) for all joints (`None`) of those envs.

Lines 113–116 What `soil.reset` does and does not do: it rewrites the default positions and zero velocities of every particle belonging to those envs (Isaac Lab's `MPMObject.reset` calls `write_nodal_state_to_sim_index` with the default nodal state). It does not clear the solver's internal history; Isaac Lab offers `NewtonMPMManager.reset_solver_state` for an exact reset, and the tricycle has not needed it.

Line 117 Reset the soil of those envs.

Line 119 Zero the stored actions for those envs, so the observation's "previous action" is zero at episode start.

Line 120 Initialise the reward's memory to the env-local spawn x (which is 0), so the first step's displacement is measured from the spawn point and not from wherever the previous episode ended.

Isaac Lab: index-based versus mask-based writes

Isaac Lab develop offers two flavours of every asset write: `write_root_pose_to_sim(data, env_ids)` (index-based, used here) and `write_root_pose_to_sim_mask(data, env_mask)` (mask-based, avoids a GPU–CPU sync when many envs reset). For a handful of envs the index form is simplest and its cost is negligible.

Chapter 12

agents/: the PPO configuration

12.1 agents/__init__.py

Role: an empty file. Its presence makes `agents` a regular Python package so that `from . import agents` in `tricycle/__init__.py` works and `setuptools` includes the folder in the editable install. 0 lines. Without it, `packages.find` skips the folder, `from . import agents` raises, and the task never registers. Creating the file is the entire fix; no reinstall is needed for it (Chapter 6).

12.2 agents/rsl_rl_ppo_cfg.py

Role: the hyper-parameters of the learner. Instantiated at configuration time (Section 3.4), converted to a plain dict and handed to `rsl_rl`'s `OnPolicyRunner`. 40 lines. The meaning of each number is in Chapter 2.

```
1  # agents/rsl_rl_ppo_cfg.py -- deliberately tiny. If reward doesn't climb in
2  # a few hundred iterations, the problem is the env, not the learner.
3  #
4  # Isaac Lab converts this "policy" style config into rsl_rl's newer actor /
5  # critic model configs at start-up (isaacclab_rl.rsl_rl.utils), so it works
6  # with the rsl_rl version the develop branch installs.
7
8  from isaaclab.utils.configclass import configclass
9  from isaaclab_rl.rsl_rl import RslRlOnPolicyRunnerCfg, RslRlPpoActorCriticCfg,
   ↪ RslRlPpoAlgorithmCfg
```

Lines 1–2 The philosophy: the learner is deliberately small so that a flat reward curve points at the environment, not at PPO.

Lines 4–6 Isaac Lab develop ships `rsl_rl` 5.x, whose native config uses separate `actor` and `critic` model configs. This file uses the older combined `policy` config; Isaac Lab's `handle_deprecated_rsl_rl_cfg` (in `source/isaacclab_rl/isaacclab_rl/rsl_rl/utils.py`) converts it at start-up into an `RslRlMLPModelCfg` for the actor and one for the critic, printing a deprecation notice. The behaviour is identical.

Line 8 The `configclass` decorator.

Line 9 The three config classes: the runner (loop settings), the actor-critic network, and the PPO algorithm.

```
12 @configclass
13 class TricyclePPORunnerCfg(RslRlOnPolicyRunnerCfg):
14     num_steps_per_env = 50          # half an episode per rollout at 50 Hz
15     max_iterations = 300
16     save_interval = 50
```

```

17     experiment_name = "tricycle"
18
19     policy = RslRlPpoActorCriticCfg(
20         init_noise_std=1.0,
21         actor_hidden_dims=[64, 64],
22         critic_hidden_dims=[64, 64],
23         activation="elu",
24         actor_obs_normalization=False,
25         critic_obs_normalization=False,
26     )

```

Line 12 A configclass ...

Line 13 ... inheriting `RslRlOnPolicyRunnerCfg`, which fixes `class_name = "OnPolicyRunner"` and provides defaults such as `seed = 42`, `device = "cuda:0"`, `logger = "tensorboard"`, `run_name = ""`, `resume = False`.

Line 14 Each iteration collects 50 steps per env before updating. At 50 Hz that is one second of driving per env, half an episode, so every rollout contains episode boundaries and the value function sees both early and late steps.

Line 15 Stop after 300 iterations. With 4 envs that is $300 \times 50 \times 4 = 60,000$ environment steps. Override with `-max_iterations`.

Line 16 Save a checkpoint every 50 iterations (`model_50.pt`, `model_100.pt`, ...) plus a final one.

Line 17 The experiment name; checkpoints land in `logs/rsl_rl/tricycle/<timestamp>/.`

Line 19 The network config.

Line 20 Initial standard deviation of the Gaussian policy: 1.0 in action units. Since actions are clamped to $[-1, 1]$, early exploration is essentially uniform over the whole range, which is what a task with an unknown good action wants.

Line 21 Actor: two hidden layers of 64 units.

Line 22 Critic: the same shape.

Line 23 ELU activations for both.

Lines 24–25 No running observation normalisation for either network. The observations are already in sensible ranges (velocities in m/s, angles in rad, actions in $[-1, 1]$). These two fields replace the deprecated runner-level `empirical_normalization = False` that this file used to set; they are what the conversion helper would have derived from it.

Line 26 End of the policy config.

```

27     algorithm = RslRlPpoAlgorithmCfg(
28         value_loss_coef=1.0,
29         use_clipped_value_loss=True,
30         clip_param=0.2,
31         entropy_coef=0.01,
32         num_learning_epochs=4,
33         num_mini_batches=2,
34         learning_rate=1.0e-3,
35         schedule="adaptive",
36         gamma=0.99,
37         lam=0.95,

```

```
38         desired_kl=0.01,  
39         max_grad_norm=1.0,  
40     )
```

Line 27 The PPO algorithm config (`class_name = "PPO"` by default).

Line 28 Weight of the critic's loss relative to the policy loss.

Line 29 Clip the value-function update as well as the policy ratio (PPO2 style), which stabilises the critic.

Line 30 The PPO clipping range $\epsilon = 0.2$.

Line 31 Entropy bonus coefficient 0.01: a mild push to keep exploring.

Line 32 Four passes over each rollout.

Line 33 Two mini-batches per pass: with $4 \text{ envs} \times 50 \text{ steps}$ the mini-batch is 100 samples.

Line 34 Initial Adam learning rate 10^{-3} .

Line 35 Adaptive schedule: the learning rate is adjusted from the measured KL divergence after each mini-batch (Chapter 2).

Line 36 Discount $\gamma = 0.99$, horizon about 100 steps = one episode.

Line 37 GAE $\lambda = 0.95$.

Line 38 Target KL of 0.01 for the adaptive schedule.

Line 39 Gradient norm clipped to 1.0.

Line 40 End.

Isaac Lab: from cfg to runner

`train_rsl_rl.py` calls `agent_cfg.to_dict()` and passes the dict to `OnPolicyRunner(env, cfg_dict, log_dir, device)`. The runner builds the actor and critic from the (converted) model configs, the PPO object from `algorithm`, and a rollout storage sized `num_envs × num_steps_per_env`. The dict is also written to `params/agent.yaml` in the log directory, so every run records the exact hyper-parameters it used.

Chapter 13

The USD asset files

Role: the car as Isaac Lab actually loads it. These files are generated (Chapter 5), so they are explained block by block rather than line by line; every block is covered. Listings are the committed files. The paths are relative to `assets/`.

13.1 tricycle/tricycle.usda: the entry point

```
1  #usda 1.0
2  (
3      customLayerData = {
4          string creator = "MuJoCo USD Converter v0.2.0"
5      }
6      defaultPrim = "tricycle"
7      doc = """Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\
      ↪ Temp\\tmpniw4ro8v\\tricycle.usdc
8
9
10     Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjcf_
      ↪ import_tricycle_df1lgh4s\\usdex_tricycle\\tricycle.usdc
11
12
13     Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjcf_
      ↪ import_tricycle_df1lgh4s\\temp_tricycle\\tricycle.usd
14     """
15     kilogramsPerUnit = 1
16     metersPerUnit = 1
17     upAxis = "Z"
18 )
19
20 def Xform "tricycle" (
21     prepend references = @./payloads/base.usda@
22     variants = {
23         string Physics = "physics"
24     }
25     append variantSets = "Physics"
26 )
27 {
28     variantSet "Physics" = {
29         "mujoco" (
30             prepend payload = @./payloads/Physics/mujoco.usda@
31         ) {
32
33         }
34         "none" {
35
36         }
37         "physics" (
38             prepend payload = @./payloads/Physics/physics.usda@
39         ) {
40
```

```

41     }
42     "physx" (
43         prepend payload = @./payloads/Physics/physx.usda@
44     ) {
45
46     }
47 }
48 }

```

Lines 1–18 Layer metadata. `creator` records the converter version; `doc` keeps the temp paths the converter went through (three stages: MuJoCo USD Converter, the `usdex` rewrite, the Isaac asset transformer); `kilogramsPerUnit/metersPerUnit` say the file is in SI units; `upAxis = "Z"` matches Isaac Lab and MuJoCo. `defaultPrim = "tricycle"` tells a referencing stage which prim to graft in, which is how `UsdFileCfg` places `/tricycle` at `{ENV_REGEX_NS}/Tricycle`.

Lines 20–27 The root prim, an `Xform`, pulls in `payloads/base.usda` by reference and declares a variant set named `Physics` whose selected variant is `"physics"`.

Lines 28–48 The four variants. `mujoco`, `physics` and `physx` each add a payload file; `none` adds nothing (a visual-only car). Only the selected variant's payload is composed, so with `"physics"` selected, `Physics/physics.usda` is loaded and the MuJoCo/PhysX-specific files are not.

13.2 payloads/base.usda: geometry

```

1  #usda 1.0
2  (
3      customLayerData = {
4          string creator = "MuJoCo USD Converter v0.2.0"
5      }
6      defaultPrim = "tricycle"
7      doc = """Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\
      ↪ Temp\\tmpni4ro8v\\tricycle.usdc
8
9
10     Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjcf_
    ↪ import_tricycle_df1lgh4s\\usdex_tricycle\\tricycle.usdc
11
12
13     Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjcf_
    ↪ import_tricycle_df1lgh4s\\temp_tricycle\\tricycle.usd
14     """
15     kilogramsPerUnit = 1
16     metersPerUnit = 1
17     subLayers = [
18         @./robot.usda@
19     ]
20     upAxis = "Z"
21 )
22
23 def Xform "tricycle" (
24     prepend apiSchemas = ["GeomModelAPI", "IsaacRobotAPI"]
25     assetInfo = {
26         string name = "tricycle"
27     }
28     kind = "component"
29 )
30 {
31     float3[] extentsHint = [(-0.25, -0.26, -1.4901161e-9), (0.32, 0.26, 0.2)]
32     custom rel isaac:physics:robotJoints
33     custom rel isaac:physics:robotLinks
34     custom token isaac:robotType = "Default"

```

```

35
36     def Scope "Geometry"
37     {
38         def Xform "chassis" (
39             prepend apiSchemas = ["IsaacLinkAPI"]
40         )
41         {
42             quatf xformOp:orient = (1, 0, 0, 0)
43             float3 xformOp:scale = (1, 1, 1)
44             double3 xformOp:translate = (0, 0, 0.14)
45             uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "xformOp:
↪ scale"]

```

Lines 1–22 Metadata as above, plus `subLayers = [ @./robot.usda@ ]`: the robot-schema layer is stacked under this one, so its `over` blocks apply to the prims defined here.

Lines 23–30 The root prim gets `GeomModelAPI` and `IsaacRobotAPI`; `kind = "component"` marks it as one model.

Line 31 `extentsHint`: the bounding box of the whole car, from the back of the rear wheels ($x = -0.25$) to the front of the front wheel ($x = 0.32$), ± 0.26 in y , and 0 to 0.2 in z (wheel bottoms to wheel tops). Unchanged by the new body because the frame lies inside this box.

Lines 32–34 Empty robot-joint/robot-link relationships and the robot type, filled in by Isaac's asset transformer.

Line 36 All geometry lives under a `Geometry` scope.

Lines 38–45 The chassis body: an `Xform` with `IsaacLinkAPI`, translated to $z = 0.14$ (the MJCF body pos), identity orientation, unit scale. `xformOpOrder` fixes the order `translate` → `orient` → `scale`.

```

47     def Cube "left_rail" (
48         prepend apiSchemas = ["PhysicsCollisionAPI", "MaterialBindingAPI"]
49     )
50     {
51         float3[] extent = [(-1, -1, -1), (1, 1, 1)]
52         rel material:binding:physics = </tricycle/Physics/PhysicsMaterial>
53         color3f[] primvars:displayColor = [(0.2, 0.4, 0.8)] (
54             interpolation = "constant"
55         )
56         int[] primvars:displayColor:indices = None
57         float[] primvars:displayOpacity = [1] (
58             interpolation = "constant"
59         )
60         int[] primvars:displayOpacity:indices = None
61         double size = 2
62         quatf xformOp:orient = (0.9857637, 0, 0, -0.16813646)
63         float3 xformOp:scale = (0.1961, 0.02, 0.015)
64         double3 xformOp:translate = (0.035, 0.065, 0)
65         uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
66     }
67
68     def Cube "right_rail" (
69         prepend apiSchemas = ["PhysicsCollisionAPI", "MaterialBindingAPI"]
70     )
71     {
72         float3[] extent = [(-1, -1, -1), (1, 1, 1)]
73         rel material:binding:physics = </tricycle/Physics/PhysicsMaterial>
74         color3f[] primvars:displayColor = [(0.2, 0.4, 0.8)] (

```

```

75         interpolation = "constant"
76     )
77     int[] primvars:displayColor:indices = None
78     float[] primvars:displayOpacity = [1] (
79         interpolation = "constant"
80     )
81     int[] primvars:displayOpacity:indices = None
82     double size = 2
83     quatf xformOp:orient = (0.9857637, 0, 0, 0.16813646)
84     float3 xformOp:scale = (0.1961, 0.02, 0.015)
85     double3 xformOp:translate = (0.035, -0.065, 0)
86     uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
87     }
88
89     def Cube "crossbar" (
90         prepend apiSchemas = ["PhysicsCollisionAPI", "MaterialBindingAPI"]
91     )
92     {
93         float3[] extent = [(-1, -1, -1), (1, 1, 1)]
94         rel material:binding:physics = </tricycle/Physics/PhysicsMaterial>
95         color3f[] primvars:displayColor = [(0.2, 0.4, 0.8)] (
96             interpolation = "constant"
97         )
98         int[] primvars:displayColor:indices = None
99         float[] primvars:displayOpacity = [1] (
100             interpolation = "constant"
101         )
102         int[] primvars:displayOpacity:indices = None
103         double size = 2
104         quatf xformOp:orient = (1, 0, 0, 0)
105         float3 xformOp:scale = (0.02, 0.15, 0.015)
106         double3 xformOp:translate = (-0.15, 0, 0)
107         uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
108     }
109
110     def Cube "deck" (
111         prepend apiSchemas = ["PhysicsCollisionAPI", "MaterialBindingAPI"]
112     )
113     {
114         float3[] extent = [(-1, -1, -1), (1, 1, 1)]
115         rel material:binding:physics = </tricycle/Physics/PhysicsMaterial>
116         color3f[] primvars:displayColor = [(0.55, 0.6, 0.7)] (
117             interpolation = "constant"
118         )
119         int[] primvars:displayColor:indices = None
120         float[] primvars:displayOpacity = [1] (
121             interpolation = "constant"
122         )
123         int[] primvars:displayOpacity:indices = None
124         double size = 2
125         quatf xformOp:orient = (1, 0, 0, 0)
126         float3 xformOp:scale = (0.06, 0.065, 0.01)
127         double3 xformOp:translate = (-0.08, 0, 0)
128         uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
129     }

```

Lines 47–66 The left rail as a Cube prim. USD's Cube is a unit cube of edge `size = 2` (so it spans $-1..1$), and the box's half-sizes are applied as `xformOp:scale = (0.1961, 0.02, 0.015)`. `xformOp:translate` is the rail centre and `xformOp:orient` is the yaw as a quaternion in (w, x, y, z) order: $(0.98576, 0, 0, -0.16814)$ is a rotation of -19.4° about z . The prim carries

PhysicsCollisionAPI (it is a collider) and MaterialBindingAPI with the binding to the shared physics material. displayColor/displayOpacity come from the MJCF rgba. extent is the unit cube's own bound; the scale is applied on top.

Lines 68–87 The right rail, identical except for the mirrored translate and the opposite quaternion sign.

Lines 89–108 The crossbar: no rotation, half-sizes (0.02,0.15,0.015) at $x = -0.15$.

Lines 110–129 The deck: half-sizes (0.06,0.065,0.01) at $x = -0.08$, in the lighter colour.

```

131     def Xform "fork" (
132         prepend apiSchemas = ["IsaacLinkAPI"]
133     )
134     {
135         quatf xformOp:orient = (1, 0, 0, 0)
136         float3 xformOp:scale = (1, 1, 1)
137         double3 xformOp:translate = (0.22, 0, -0.04)
138         uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
139
140     def Capsule "Capsule" (
141         prepend apiSchemas = ["PhysicsCollisionAPI", "MaterialBindingAPI"]
142     )
143     {
144         uniform token axis = "Z"
145         float3[] extent = [(-0.012, -0.012, -0.032), (0.012, 0.012, 0.032)]
146         double height = 0.04
147         rel material:binding:physics = </tricycle/Physics/PhysicsMaterial>
148         color3f[] primvars:displayColor = [(0.5, 0.5, 0.5)] (
149             interpolation = "constant"
150         )
151         int[] primvars:displayColor:indices = None
152         float[] primvars:displayOpacity = [1] (
153             interpolation = "constant"
154         )
155         int[] primvars:displayOpacity:indices = None
156         double radius = 0.012
157         quatf xformOp:orient = (6.123234e-17, 1, 0, 0)
158         float3 xformOp:scale = (1, 1, 1)
159         double3 xformOp:translate = (0, 0, 0)
160         uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
161     }
162
163     def Xform "front_wheel_body" (
164         prepend apiSchemas = ["IsaacLinkAPI"]
165     )
166     {
167         quatf xformOp:orient = (1, 0, 0, 0)
168         float3 xformOp:scale = (1, 1, 1)
169         double3 xformOp:translate = (0, 0, 0)
170         uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
171
172     def Sphere "Sphere" (
173         prepend apiSchemas = ["PhysicsCollisionAPI", "MaterialBindingAPI"]
174     )
175     {
176         float3[] extent = [(-0.1, -0.1, -0.1), (0.1, 0.1, 0.1)]
177         rel material:binding:physics = </tricycle/Physics/PhysicsMaterial>
178         color3f[] primvars:displayColor = [(0.15, 0.15, 0.15)] (
179             interpolation = "constant"
180         )

```

```

181         int[] primvars:displayColor:indices = None
182         float[] primvars:displayOpacity = [1] (
183             interpolation = "constant"
184         )
185         int[] primvars:displayOpacity:indices = None
186         double radius = 0.1
187         quatf xformOp:orient = (1, 0, 0, 0)
188         float3 xformOp:scale = (1, 1, 1)
189         double3 xformOp:translate = (0, 0, 0)
190         uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient",
↪ "xformOp:scale"]
191     }
192 }
193 }

```

Lines 131–138 The fork body under the chassis, translated to (0.22, 0, −0.04): the MJCF body pos.

Lines 140–161 The fork’s capsule. USD capsules are along an axis (axis = "Z") with height being the cylinder part (0.04) and radius 0.012. The odd orient = (6.1e-17, 1, 0, 0) is a 180° flip about *x*: the MJCF fromto pointed downwards and the converter preserved that direction exactly.

Lines 163–193 The front wheel body nested inside the fork (at the fork origin), containing a Sphere of radius 0.1, coloured dark grey, with the same collision and material tags.

```

195     def Xform "rear_left_body" (
196         prepend apiSchemas = ["IsaacLinkAPI"]
197     )
198     {
199         quatf xformOp:orient = (1, 0, 0, 0)
200         float3 xformOp:scale = (1, 1, 1)
201         double3 xformOp:translate = (-0.15, 0.16, -0.04)
202         uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
203
204         def Sphere "Sphere" (
205             prepend apiSchemas = ["PhysicsCollisionAPI", "MaterialBindingAPI"]
206         )
207         {
208             float3[] extent = [(-0.1, -0.1, -0.1), (0.1, 0.1, 0.1)]
209             rel material:binding:physics = </tricycle/Physics/PhysicsMaterial>
210             color3f[] primvars:displayColor = [(0.15, 0.15, 0.15)] (
211                 interpolation = "constant"
212             )
213             int[] primvars:displayColor:indices = None
214             float[] primvars:displayOpacity = [1] (
215                 interpolation = "constant"
216             )
217             int[] primvars:displayOpacity:indices = None
218             double radius = 0.1
219             quatf xformOp:orient = (1, 0, 0, 0)
220             float3 xformOp:scale = (1, 1, 1)
221             double3 xformOp:translate = (0, 0, 0)
222             uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
223         }
224     }
225
226     def Xform "rear_right_body" (
227         prepend apiSchemas = ["IsaacLinkAPI"]
228     )

```

```

229         {
230             quatf xformOp:orient = (1, 0, 0, 0)
231             float3 xformOp:scale = (1, 1, 1)
232             double3 xformOp:translate = (-0.15, -0.16, -0.04)
233             uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
234
235             def Sphere "Sphere" (
236                 prepend apiSchemas = ["PhysicsCollisionAPI", "MaterialBindingAPI"]
237             )
238             {
239                 float3[] extent = [(-0.1, -0.1, -0.1), (0.1, 0.1, 0.1)]
240                 rel material:binding:physics = </tricycle/Physics/PhysicsMaterial>
241                 color3f[] primvars:displayColor = [(0.15, 0.15, 0.15)] (
242                     interpolation = "constant"
243                 )
244                 int[] primvars:displayColor:indices = None
245                 float[] primvars:displayOpacity = [1] (
246                     interpolation = "constant"
247                 )
248                 int[] primvars:displayOpacity:indices = None
249                 double radius = 0.1
250                 quatf xformOp:orient = (1, 0, 0, 0)
251                 float3 xformOp:scale = (1, 1, 1)
252                 double3 xformOp:translate = (0, 0, 0)
253                 uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:orient", "
↪ xformOp:scale"]
254             }
255         }
256     }
257 }
258
259 def Scope "Physics"
260 {
261     def Material "PhysicsMaterial"
262     {
263     }
264 }
265 }

```

Lines 195–225 The left rear wheel body at $(-0.15, 0.16, -0.04)$ with its sphere.

Lines 226–255 The right rear wheel body at $(-0.15, -0.16, -0.04)$ with its sphere.

Lines 259–266 A Physics scope holding the one shared PhysicsMaterial that all colliders bind to. Its attributes are added by `physics.usda`.

Isaac Lab: what Newton builds from this

Reading the composed stage, Isaac Lab's Newton backend creates one Newton body per prim with `PhysicsRigidBodyAPI` (chassis, fork, three wheels), one Newton shape per prim with `PhysicsCollisionAPI` (four cubes, one capsule, three spheres), and one joint per `PhysicsRevoluteJoint` prim. Body mass and inertia are accumulated from the shapes' `physics:mass` and geometry, exactly as MuJoCo did when it compiled the MJCF.

13.3 payloads/robot.usda: robot schema tags

```

1  #usda 1.0
2  (
3      customLayerData = {

```

```

4     string creator = "MuJoCo USD Converter v0.2.0"
5 }
6 defaultPrim = "tricycle"
7 doc = """Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\
    ↪ Temp\\tmpnix4ro8v\\tricycle.usdc
8
9
10 Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjc_
    ↪ import_tricycle_df1lgh4s\\usdex_tricycle\\tricycle.usdc
11
12
13 Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjc_
    ↪ import_tricycle_df1lgh4s\\temp_tricycle\\tricycle.usd
14 """
15 kilogramsPerUnit = 1
16 metersPerUnit = 1
17 upAxis = "Z"
18 )
19
20 over "tricycle" (
21     prepend apiSchemas = ["IsaacRobotAPI"]
22 )
23 {
24     custom rel isaac:physics:robotJoints
25     custom rel isaac:physics:robotLinks
26
27     over "Geometry"
28     {
29         over "chassis" (
30             prepend apiSchemas = ["IsaacLinkAPI"]
31         )
32         {
33             over "fork" (
34                 prepend apiSchemas = ["IsaacLinkAPI"]
35             )
36             {
37                 over "steer" (
38                     prepend apiSchemas = ["IsaacJointAPI"]
39                 )
40                 {
41                 }
42
43                 over "front_wheel_body" (
44                     prepend apiSchemas = ["IsaacLinkAPI"]
45                 )
46                 {
47                     over "front_wheel" (
48                         prepend apiSchemas = ["IsaacJointAPI"]
49                     )
50                     {
51                     }
52                 }
53             }
54
55             over "rear_left_body" (
56                 prepend apiSchemas = ["IsaacLinkAPI"]
57             )
58             {
59                 over "rear_left" (
60                     prepend apiSchemas = ["IsaacJointAPI"]
61                 )
62                 {
63                 }
64             }

```

```

65
66         over "rear_right_body" (
67             prepend apiSchemas = ["IsaacLinkAPI"]
68         )
69         {
70             over "rear_right" (
71                 prepend apiSchemas = ["IsaacJointAPI"]
72             )
73             {
74             }
75         }
76     }
77 }
78 }

```

Lines 1–19 Metadata.

Lines 20–25 over "tricycle": adds IsaacRobotAPI and the two relationship attributes that list the robot's joints and links.

Lines 27–78 A mirror of the body tree in which every body gets IsaacLinkAPI and every joint gets IsaacJointAPI. These tags are how Isaac Lab tools recognise “this prim is a robot link/joint” independent of the physics variant. Note the joint prims (**steer**, **front_wheel**, **rear_left**, **rear_right**) live under the *child* body of the joint, which is also where **physics.usda** defines them.

13.4 payloads/Physics/physics.usda: bodies, joints, masses

```

1  #usda 1.0
2  (
3      customLayerData = {
4          string creator = "MuJoCo USD Converter v0.2.0"
5      }
6      defaultPrim = "tricycle"
7      doc = """Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\
      ↪ Temp\\tmpniw4ro8v\\tricycle.usdc
8
9
10     Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjc_
      ↪ import_tricycle_df1lgh4s\\usdex_tricycle\\tricycle.usdc
11
12
13     Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjc_
      ↪ import_tricycle_df1lgh4s\\temp_tricycle\\tricycle.usd
14     """
15     kilogramsPerUnit = 1
16     metersPerUnit = 1
17     upAxis = "Z"
18 )
19
20 over "tricycle"
21 {
22     over "Geometry"
23     {
24         over "chassis" (
25             prepend apiSchemas = ["PhysicsRigidBodyAPI", "PhysicsArticulationRootAPI", "
      ↪ NewtonArticulationRootAPI"]
26         )
27         {
28             bool newton:selfCollisionEnabled = 0
29

```

```

30         over "left_rail" (
31             prepend apiSchemas = ["NewtonCollisionAPI", "PhysicsMassAPI"]
32         )
33         {
34             float newton:contactGap = 0
35             float physics:density = 1000
36             float physics:mass = 0.8
37         }
38
39         over "right_rail" (
40             prepend apiSchemas = ["NewtonCollisionAPI", "PhysicsMassAPI"]
41         )
42         {
43             float newton:contactGap = 0
44             float physics:density = 1000
45             float physics:mass = 0.8
46         }
47
48         over "crossbar" (
49             prepend apiSchemas = ["NewtonCollisionAPI", "PhysicsMassAPI"]
50         )
51         {
52             float newton:contactGap = 0
53             float physics:density = 1000
54             float physics:mass = 0.8
55         }
56
57         over "deck" (
58             prepend apiSchemas = ["NewtonCollisionAPI", "PhysicsMassAPI"]
59         )
60         {
61             float newton:contactGap = 0
62             float physics:density = 1000
63             float physics:mass = 2.6
64         }

```

Lines 1–18 Metadata.

Lines 20–22 Overrides on the existing prims.

Lines 24–28 The chassis becomes a rigid body (`PhysicsRigidBodyAPI`), the articulation root (`PhysicsArticulationRootAPI`, `NewtonArticulationRootAPI`), and `newton:selfCollisionEnabled = 0`: bodies of the car do not collide with each other. This is why the rails can end inside the wheel spheres and the fork capsule can sit inside the front wheel without generating contacts.

Lines 30–37 The left rail gets `NewtonCollisionAPI` (Newton-specific collision settings; `newton:contactGap = 0`) and `PhysicsMassAPI` with `physics:mass = 0.8`. `physics:density = 1000` is written by the converter but mass takes precedence.

Lines 39–64 The same for the right rail (0.8), crossbar (0.8) and deck (2.6). The four masses sum to 5.0 kg.

```

66         over "fork" (
67             prepend apiSchemas = ["PhysicsRigidBodyAPI"]
68         )
69         {
70             over "Capsule" (
71                 prepend apiSchemas = ["NewtonCollisionAPI", "PhysicsMassAPI"]
72             )
73             {
74                 float newton:contactGap = 0
75                 float physics:density = 1000

```

```

76         float physics:mass = 0.1
77     }
78
79     over "front_wheel_body" (
80         prepend apiSchemas = ["PhysicsRigidBodyAPI"]
81     )
82     {
83         over "Sphere" (
84             prepend apiSchemas = ["NewtonCollisionAPI", "PhysicsMassAPI"]
85         )
86         {
87             float newton:contactGap = 0
88             float physics:density = 1000
89             float physics:mass = 0.5
90         }
91
92         def PhysicsRevoluteJoint "front_wheel" (
93             prepend apiSchemas = ["PhysicsDriveAPI:angular", "
↪ PhysicsJointStateAPI:angular"]
94         )
95         {
96             uniform token physics:axis = "Y"
97             custom rel physics:body0
98             prepend rel physics:body0 = </tricycle/Geometry/chassis/fork>
99             custom rel physics:body1
100             prepend rel physics:body1 = </tricycle/Geometry/chassis/fork/front_
↪ wheel_body>
101             point3f physics:localPos0 = (0, 0, 0)
102             point3f physics:localPos1 = (0, 0, 0)
103             quatf physics:localRot0 = (1, 0, 0, 0)
104             quatf physics:localRot1 = (1, 0, 0, 0)
105         }
106     }
107
108     def PhysicsRevoluteJoint "steer" (
109         prepend apiSchemas = ["PhysicsDriveAPI:angular", "PhysicsJointStateAPI:
↪ angular"]
110     )
111     {
112         uniform token physics:axis = "Z"
113         custom rel physics:body0
114         prepend rel physics:body0 = </tricycle/Geometry/chassis>
115         custom rel physics:body1
116         prepend rel physics:body1 = </tricycle/Geometry/chassis/fork>
117         point3f physics:localPos0 = (0.22, 0, -0.04)
118         point3f physics:localPos1 = (0, 0, 0)
119         quatf physics:localRot0 = (1, 0, 0, 0)
120         quatf physics:localRot1 = (1, 0, 0, 0)
121         float physics:lowerLimit = -34.37747
122         float physics:upperLimit = 34.37747
123     }
124 }

```

Lines 66–68 The fork becomes a rigid body.

Lines 70–76 Its capsule: collision tags and 0.1 kg.

Lines 78–80 The front wheel body becomes a rigid body.

Lines 82–89 Its sphere: collision tags and 0.5 kg.

Lines 92–105 The `front_wheel` joint: a `PhysicsRevoluteJoint` about axis Y, between `body0` (the fork) and `body1` (the wheel body), located at the origin of both (`localPos0/1 = 0`, identity

rotations). It carries `PhysicsDriveAPI:angular` and `PhysicsJointStateAPI:angular`, the schemas through which Isaac Lab sets drive targets/gains and reads joint state. No limits: it spins freely.

Lines 108–124 The `steer` joint: revolute about Z, between the chassis (`body0`) and the fork (`body1`), anchored at `localPos0 = (0.22, 0, -0.04)` in the chassis and at the fork's origin. Limits $\pm 34.377^\circ$ are the MJCF's ± 0.6 rad converted to degrees, which is USD's unit for joint limits.

```

126         over "rear_left_body" (
127             prepend apiSchemas = ["PhysicsRigidBodyAPI"]
128         )
129     {
130         over "Sphere" (
131             prepend apiSchemas = ["NewtonCollisionAPI", "PhysicsMassAPI"]
132         )
133         {
134             float newton:contactGap = 0
135             float physics:density = 1000
136             float physics:mass = 0.5
137         }
138
139         def PhysicsRevoluteJoint "rear_left" (
140             prepend apiSchemas = ["PhysicsDriveAPI:angular", "PhysicsJointStateAPI:
↪ angular"]
141         )
142         {
143             uniform token physics:axis = "Y"
144             custom rel physics:body0
145             prepend rel physics:body0 = </tricycle/Geometry/chassis>
146             custom rel physics:body1
147             prepend rel physics:body1 = </tricycle/Geometry/chassis/rear_left_body>
148             point3f physics:localPos0 = (-0.15, 0.16, -0.04)
149             point3f physics:localPos1 = (0, 0, 0)
150             quatf physics:localRot0 = (1, 0, 0, 0)
151             quatf physics:localRot1 = (1, 0, 0, 0)
152         }
153     }
154
155     over "rear_right_body" (
156         prepend apiSchemas = ["PhysicsRigidBodyAPI"]
157     )
158     {
159         over "Sphere" (
160             prepend apiSchemas = ["NewtonCollisionAPI", "PhysicsMassAPI"]
161         )
162         {
163             float newton:contactGap = 0
164             float physics:density = 1000
165             float physics:mass = 0.5
166         }
167
168         def PhysicsRevoluteJoint "rear_right" (
169             prepend apiSchemas = ["PhysicsDriveAPI:angular", "PhysicsJointStateAPI:
↪ angular"]
170         )
171         {
172             uniform token physics:axis = "Y"
173             custom rel physics:body0
174             prepend rel physics:body0 = </tricycle/Geometry/chassis>
175             custom rel physics:body1
176             prepend rel physics:body1 = </tricycle/Geometry/chassis/rear_right_body>
177             point3f physics:localPos0 = (-0.15, -0.16, -0.04)

```

```

178         point3f physics:localPos1 = (0, 0, 0)
179         quatf physics:localRot0 = (1, 0, 0, 0)
180         quatf physics:localRot1 = (1, 0, 0, 0)
181     }
182 }
183 }
184 }
185
186 over "Physics"
187 {
188     over "PhysicsMaterial" (
189         prepend apiSchemas = ["PhysicsMaterialAPI", "NewtonMaterialAPI"]
190     )
191     {
192         float newton:rollingFriction = 0.0001
193         float newton:torsionalFriction = 0.005
194         float physics:dynamicFriction = 0.8
195     }
196 }
197 }

```

Lines 126–153 The left rear wheel: rigid body, 0.5 kg sphere, and the `rear_left` revolute joint about Y between chassis and wheel, anchored at $(-0.15, 0.16, -0.04)$ in the chassis frame. No limits.

Lines 155–183 The right rear wheel and `rear_right` joint, mirrored.

Lines 186–197 The shared physics material gets `PhysicsMaterialAPI` and `NewtonMaterialAPI` with the friction values from the MJCF default geom: dynamic 0.8, torsional 0.005, rolling 0.0001.

Isaac Lab: how the env cfg's actuators meet these joints

`ImplicitActuatorCfg` entries in `tricycle_env_cfg.py` are matched to these joint prims by name (`joint_names_expr`). At start-up Isaac Lab writes the configured stiffness, damping and limits into the joints' drive attributes (or, with `use_newton_actuators`, into native Newton actuators). The USD carries no gains of its own, which is why the cfg must supply them.

13.5 payloads/Physics/mujoco.usda: MuJoCo variant

```

1  #usda 1.0
2  (
3      customLayerData = {
4          string creator = "MuJoCo USD Converter v0.2.0"
5      }
6      defaultPrim = "tricycle"
7      doc = """Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\
      ↪ Temp\\tmpnix4ro8v\\tricycle.usdc
8
9
10     Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjc_
      ↪ import_tricycle_df1lgh4s\\usdex_tricycle\\tricycle.usdc
11
12
13     Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjc_
      ↪ import_tricycle_df1lgh4s\\temp_tricycle\\tricycle.usd
14     """
15     kilogramsPerUnit = 1
16     metersPerUnit = 1
17     subLayers = [

```

```

18     @./physics.usda@
19 ]
20 upAxis = "Z"
21 )
22
23 over "tricycle"
24 {
25     over "Geometry"
26     {
27         over "chassis"
28         {
29             over "left_rail" (
30                 prepend apiSchemas = ["MjcCollisionAPI"]
31             )
32             {
33             }
34
35             over "right_rail" (
36                 prepend apiSchemas = ["MjcCollisionAPI"]
37             )
38             {
39             }
40
41             over "crossbar" (
42                 prepend apiSchemas = ["MjcCollisionAPI"]
43             )
44             {
45             }
46
47             over "deck" (
48                 prepend apiSchemas = ["MjcCollisionAPI"]
49             )
50             {
51             }
52
53             over "fork"
54             {
55                 over "Capsule" (
56                     prepend apiSchemas = ["MjcCollisionAPI"]
57                 )
58                 {
59                 }
60
61                 over "front_wheel_body"
62                 {
63                     over "Sphere" (
64                         prepend apiSchemas = ["MjcCollisionAPI"]
65                     )
66                     {
67                     }
68
69                     over "front_wheel" (
70                         delete apiSchemas = ["PhysicsDriveAPI:angular", "PhysicsJointStateAPI:
↪ angular"]
71                         prepend apiSchemas = ["MjcJointAPI"]
72                     )
73                     {
74                         uniform double mjc:damping = 0.02
75                     }
76                 }
77
78                 over "steer" (
79                     delete apiSchemas = ["PhysicsDriveAPI:angular", "PhysicsJointStateAPI:
↪ angular"]

```

```

80         prepend apiSchemas = ["MjcJointAPI"]
81     )
82     {
83         uniform double mjc:damping = 0.02
84     }
85 }
86
87 over "rear_left_body"
88 {
89     over "Sphere" (
90         prepend apiSchemas = ["MjcCollisionAPI"]
91     )
92     {
93     }
94
95     over "rear_left" (
96         delete apiSchemas = ["PhysicsDriveAPI:angular", "PhysicsJointStateAPI:
↪ angular"]
97         prepend apiSchemas = ["MjcJointAPI"]
98     )
99     {
100         uniform double mjc:damping = 0.02
101     }
102 }
103
104 over "rear_right_body"
105 {
106     over "Sphere" (
107         prepend apiSchemas = ["MjcCollisionAPI"]
108     )
109     {
110     }
111
112     over "rear_right" (
113         delete apiSchemas = ["PhysicsDriveAPI:angular", "PhysicsJointStateAPI:
↪ angular"]
114         prepend apiSchemas = ["MjcJointAPI"]
115     )
116     {
117         uniform double mjc:damping = 0.02
118     }
119 }
120 }
121 }
122
123 over "Physics"
124 {
125     over "PhysicsMaterial" (
126         prepend apiSchemas = ["MjcMaterialAPI"]
127     )
128     {
129     }
130 }
131 }

```

Lines 1–21 Metadata; this layer sublayers `physics.usda`, so it is “physics plus MuJoCo extras”.

Lines 23–121 For every collider, `MjcCollisionAPI`; for every joint, `MjcJointAPI` with `mjc:damping = 0.02` (the MJCF default joint damping), and the generic drive/state schemas deleted because MuJoCo would read its own. This variant would be used to round-trip the asset back into a MuJoCo-native pipeline; Isaac Lab’s Newton backend uses the `physics` variant instead, so *the 0.02 joint damping from the MJCF is not active during training*. The

wheels' damping in training comes from the actuator configs (and the free front wheel has none beyond solver defaults).

Lines 123–131 MjcMaterialAPI on the shared material.

13.6 payloads/Physics/physx.usda: PhysX variant

```

1  #usda 1.0
2  (
3      customLayerData = {
4          string creator = "MuJoCo USD Converter v0.2.0"
5      }
6      defaultPrim = "tricycle"
7      doc = """Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\
      ↪ Temp\\tmpniw4ro8v\\tricycle.usdc
8
9
10     Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjc_
      ↪ import_tricycle_df1lgh4s\\usdex_tricycle\\tricycle.usdc
11
12
13     Generated from Composed Stage of root layer C:\\Users\\hanse\\AppData\\Local\\Temp\\mjc_
      ↪ import_tricycle_df1lgh4s\\temp_tricycle\\tricycle.usd
14     """
15     kilogramsPerUnit = 1
16     metersPerUnit = 1
17     subLayers = [
18         @./physics.usda@
19     ]
20     upAxis = "Z"
21 )
22
23 over "tricycle"
24 {
25     over "Geometry"
26     {
27         over "chassis"
28         {
29             over "fork"
30             {
31                 over "front_wheel_body"
32                 {
33                     over "front_wheel" (
34                         prepend apiSchemas = ["PhysxJointAPI"]
35                     )
36                     {
37                     }
38                 }
39
40                 over "steer" (
41                     prepend apiSchemas = ["PhysxJointAPI"]
42                 )
43                 {
44                 }
45             }
46
47             over "rear_left_body"
48             {
49                 over "rear_left" (
50                     prepend apiSchemas = ["PhysxJointAPI"]
51                 )
52                 {
53                 }

```

```

54         }
55
56         over "rear_right_body"
57         {
58             over "rear_right" (
59                 prepend apiSchemas = ["PhysxJointAPI"]
60             )
61             {
62             }
63         }
64     }
65 }
66 }

```

Lines 1–21 Metadata; also sublayers `physics.usda`.

Lines 23–66 Adds `PhysxJointAPI` to the four joints. Not loaded with Newton.

13.7 config.yaml and .asset_hash

```

1  asset_path: C:\Users\hanse\AppData\Local\Temp\luna_hifi_assets\tricycle.xml
2  usd_dir: C:\Users\hanse\Documents\Luna_HiFi\assets
3  usd_file_name: tricycle\tricycle.usda
4  force_usd_conversion: true
5  make_instanceable: true
6  physics_variant: physics
7  merge_mesh: false
8  collision_from_visuals: false
9  collision_type: Convex Hull
10 self_collision: false
11 import_physics_scene: false
12 fix_base: false
13 link_density: 0.0
14 robot_type: Default
15 override_gain_type: null
16 override_bias_type: null
17 override_gain_prm: null
18 override_bias_prm: null
19 run_asset_transformer: true
20 run_multi_physics_conversion: true
21 debug_mode: false
22 ##
23 # Generated by MjcfConverter on 2026-09-02 at 23:10:58.
24 ##

```

Lines 1–3 The converter's inputs: source MJCF (the temp file `write_mjcf` produced), the output directory and the output file name (with the subfolder the converter chose).

Line 4 `force_usd_conversion: true`: reconvert even if the hash matches.

Line 5 `make_instanceable: true`: mark the asset so that many copies share geometry.

Line 6 Which physics variant to select: `physics`.

Lines 7–9 Mesh handling: keep separate meshes, no collision-from-visuals, convex-hull collision for meshes (irrelevant for primitives).

Line 10 `self_collision: false`: the source of `newton:selfCollisionEnabled = 0`.

Line 11 Do not import a MuJoCo physics scene (Isaac Lab creates its own).

Line 12 The car is not fixed to the world (it has a free joint).

Line 13 No default density override.

Lines 14–18 Robot type and optional actuator gain/bias overrides, all defaults.

Lines 19–20 The two post-processing passes (Chapter 5).

Line 21 Debug off.

Lines 22–24 A trailer with the conversion date.

`.asset_hash` holds one MD5 string computed from the converter config and the MJCF contents. Isaac Lab compares it with a fresh hash on the next conversion and skips the work when they match (unless `force_usd_conversion` is set). It is not consulted at training time.

Chapter 14

The runbook commands, flag by flag

RUNBOOK.md lists the commands that work. This chapter says what each one does, so that when one fails you know which layer failed.

14.1 One-time setup

```
winget install --id Python.Python.3.12 -e
py -0
```

Installs CPython 3.12 from the Windows package manager and lists the installed interpreters (`-V:3.12` must appear). 3.10 fails twice over: Newton’s type annotations use `X | None` at run time, and the only `imgui-bundle` wheel for 3.10 on Windows is too old, so pip tries to compile it from source and fails.

```
git clone -b develop https://github.com/isaac-sim/IsaacLab.git
py -3.12 -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
.\isaacclab.bat -i
.\isaacclab.bat -i isaacsim
```

- `-b develop`: the branch with the Newton backend and the `isaacclab train` CLI. `main` does not have them.
- `py -3.12 -m venv .venv`: a virtual environment inside the Isaac Lab folder; `isaacclab.bat` looks for it there.
- `Set-ExecutionPolicy ... Bypass`: lets PowerShell run the activation script in this window only.
- `.\isaacclab.bat -i`: installs Isaac Lab’s source packages (`isaacclab`, `isaacclab_newton`, `isaacclab_rl`, `isaacclab_tasks`, `isaacclab_contrib`, ...) in editable mode plus Newton, Warp, torch and `rsl_rl`. This is the “kit-less” install: no Isaac Sim application.
- `-i isaacsim`: adds the Isaac Sim pip package, needed *only* by the MJCF converter (Chapter 5). Training never starts it.

```
.\isaacclab.bat train --rl_library rsl_rl --task Isaac-Cartpole-Direct --num_envs 16
  ↪ --max_iterations 10 physics=newton_mjwarp --viz newton
```

A smoke test with a built-in task before any of our code is involved. `physics=newton_mjwarp` is a Hydra typed selector that replaces the task’s physics section with the Newton preset (Cartpole defaults to `PhysX`); our task does not need it because `__post_init__` builds its own `NewtonCfg`. The pyglet `AssertionError` printed after “Training time” is the viewer window being torn down while a redraw is pending; it is harmless.

```

.\isaaclab.bat -p -m pip install -e C:\Users\hanse\Documents\Luna_HiFi
.\isaaclab.bat -p -c "import luna_hifi_tasks, gymnasium as gym; print([k for k in
    ↪ gym.registry if 'Luna-' in k])"

```

`-p` runs the venv’s Python with the arguments that follow. The first line installs our package in editable mode (Chapter 6); the second imports it and prints every registry key containing `Luna-`. The expected output is `['Luna-Tricycle-Direct']`. An empty list means the import ran but registered nothing, which is the missing-`__init__.py` symptom; an `ImportError` means the editable install did not take.

```

mkdir C:\Users\hanse\Documents\Luna_HiFi\assets
.\isaaclab.bat -p -c "from luna_hifi_tasks.tricycle.tricycle import write_mjcf;
    ↪ print(write_mjcf())"
.\isaaclab.bat -p scripts\tools\convert_mjcf.py C:\Users\hanse\AppData\Local\Temp\
    ↪ luna_hifi_assets\tricycle.xml C:\Users\hanse\Documents\Luna_HiFi\assets\
    ↪ tricycle.usd

```

The conversion. Line two calls the function from Chapter 9 and prints the temp path it wrote. Line three runs Isaac Lab’s converter script on that file: positional arguments are the MJCF input and the USD output. The converter ignores the exact output filename and writes `assets\tricycle\tricycle.usda` plus the payload files (Chapter 13); the env cfg points at that real location. Run these two lines again whenever `tricycle.py` changes.

14.2 Every session

```

cd C:\Users\hanse\Documents\IsaacLab
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1

```

`isaaclab.bat` must be run from the Isaac Lab folder (hence the `.\` prefix and the `cd`), and the venv must be active in each new window; “No module named ‘isaaclab_rl’” means it is not.

```

.\isaaclab.bat train --rl_library rsl_rl --task Luna-Tricycle-Direct --num_envs 4

```

Chapter 3 end to end. Headless is the default; there is no `-headless` flag on develop. Add `-viz newton` to watch, and `-max_iterations N` to override the 300 in the agent cfg. Checkpoints and TensorBoard logs go to `logs\rsl_rl\tricycle\<timestamp>\`.

```

.\isaaclab.bat play --rl_library rsl_rl --task Luna-Tricycle-Direct --num_envs 1 --
    ↪ checkpoint latest --viz newton

```

Section 3.9. `-checkpoint latest` picks the newest run’s last saved model; a path to a `model_N.pt` also works. `-num_envs 1` keeps the soil grid small; `-viz newton` opens the window in which particles are drawn because of our `play_mode()`.

14.3 Troubleshooting table, mapped to the code

Symptom	Where in this manual
<code>TypeError: Union[arg, ...]</code> on <code>newton</code> import	Python 3.10; Section 1 above.
<code>NameNotFound</code> on the task id	Package not installed, or <code>pyproject.toml</code> edited without reinstalling: Chapter 6, Section 3.3.
<code>gym</code> registry empty after import	Missing <code>__init__.py</code> ; add it, no reinstall: Chapters 6, 12.
<code>A prim already exists at path</code>	Assets constructed in <code>_setup_scene</code> : Chapter 11.
capacity exceeded for upper/lower/leaf	Raise that cap, keep the ordering: <code>cfg</code> lines 257–260, Section 4.3.
<code>Failed to create volume + CUDA 700</code>	Caps too high for memory; one failure, then the context is poisoned: Section 4.3.
<code>Mean episode length: 1.00</code>	A termination fires on step 1: check the flip sign and drift threshold, Chapter 11.
reward explodes to 10^8	Physics diverged; the invalid-state termination and reward clamp exist for this: Chapter 11.
soil invisible	<code>show_particles</code> defaults to <code>False</code> : <code>cfg</code> line 295.
car looks like the old box	Stale USD; reconvert: Chapter 5.

After any CUDA error 700, check `nvidia-smi` and kill leftover `python.exe` processes before retrying: the CUDA context is unusable and the VRAM may still be held.

Part III

Appendices

Appendix A

Glossary

Action The policy’s output for one step; here two numbers in $[-1, 1]$.

Actuator (implicit) A PD drive computed inside the physics solver from position/velocity targets and gains. `ImplicitActuatorCfg`.

Advantage How much better an action turned out than the critic expected. Estimated with GAE.

Articulation A tree of rigid bodies connected by joints, with one root. The car. `Articulation / ArticulationCfg`.

Asset Anything the scene spawns and wraps in a Python object: articulation, rigid object, MPM object, or a plain `AssetBaseCfg`.

Checkpoint A saved snapshot of the networks, `model_N.pt`.

configclass Isaac Lab’s dataclass wrapper used for every config.

Coupler Newton’s experimental solver that runs several solvers on disjoint parts of one model and links them. `CouplerProxyCfg`.

Coupler entry One named sub-solver and the bodies/particles it owns. `CouplerEntryCfg`.

Critic / value function The network that predicts expected return from an observation.

CUDA graph A recorded sequence of GPU kernels replayed without CPU overhead; requires fixed buffer sizes.

Decimation Physics steps per control step. 2 here.

DirectRLEnv Isaac Lab’s base class for environments written as explicit methods (as opposed to the manager-based workflow).

Entry point Installed package metadata that Isaac Lab uses to discover task packages (`isaacclab.tasks` group).

Env / environment instance One copy of the scene on the env grid.

Env origin World position of an env’s local frame. `scene.env_origins`.

Episode A run from reset to done; 2 s or an early termination.

GAE Generalised Advantage Estimation, parameter λ .

Hydra The config-override system behind `physics=NAME` and `env.xxx=value` command-line arguments.

InteractiveScene Turns a scene cfg into spawned prims and asset objects; owns `env_origins`.

Iteration One PPO cycle: rollout, advantage, update.

Kinematic body A rigid body with a pose but no dynamics; the slab.

Lagged coupling Proxy poses sent forward each step, feedback impulses applied one step late.

MJCF MuJoCo’s XML model format. `tricycle.py` holds one.

MJWarp MuJoCo re-implemented in Warp; Newton’s rigid-body solver.

MPM Material Point Method: particles plus a background grid for continua such as soil.

MPMObject Isaac Lab asset wrapping one particle cloud per env.

Newton NVIDIA’s GPU physics engine on Warp, Isaac Lab’s default backend on develop.

Observation The policy’s input; 15 numbers here.

Policy / actor The network mapping observations to an action distribution.

PPO Proximal Policy Optimisation, the learning algorithm.

Prim A node in a USD stage. Prim paths look like `/World/envs/env_0/Tricycle`.

Proxy A ghost copy of a body from one coupler entry placed inside another so they can interact.

Rigid object A single rigid body asset. `RigidObjectCfg`.

Rollout The batch of experience collected before an update.

Root state Position, quaternion, linear and angular velocity of an articulation’s root body: 13 numbers.

rsl_rl The learning library (ETH Zurich Robotic Systems Lab).

SimulationContext The Isaac Lab object that owns the stage and the physics manager.

Sparse grid MPM grid that allocates cells only near particles; sized by the four capacity caps.

Time-out Episode cut by the clock, not by failure; bootstrapped by the learner.

USD Universal Scene Description, the scene file format Isaac Lab builds physics from.

Warp NVIDIA’s Python-to-CUDA kernel language that Newton and Isaac Lab are written in.

Appendix B

What changed in this revision

All changes are on the branch named on the title page. None changes the task's observation, action, reward or termination definitions.

The car body

- `tricycle.py`: the single box chassis became a triangular frame (two rails, crossbar, deck). Wheel positions, radii, masses, joint names and body names are unchanged; the chassis total mass is still 5.0 kg. Geometry is computed from a few named constants and a `_rail` helper. The import-time `write_mjcf()` call was removed.
- `assets/tricycle/`: the committed USD was regenerated to match. The four new `Cube` prims replace `chassis_geom` in `base.usda`, `physics.usda` and `mujoco.usda`. The edit was validated by composing the stage with OpenUSD: every prim carries the same schemas as before and the four masses sum to 5.0 kg. The new MJCF was also compiled with MuJoCo and driven for several seconds on a plane (stands, drives straight, turns without tipping).
- If anything ever looks wrong, the converter is the source of truth: rerun the two conversion commands.

Code clean-up (behaviour preserved)

- `tricycle_env_cfg.py`: the hard-coded Windows asset path became a path relative to the file; stale comments about `MjcfFileCfg` were removed; the strip comment now states the real particle count.
- `tricycle_env.py`: removed a stale `VERIFY` marker on an import; type annotations on `self.car/self.soil`; the unreachable `_ALL_INDICES` fallback replaced by `torch.arange`; the `soil.reset` comment now states what the call does.
- `tricycle/__init__.py`: header shows the current CLI commands instead of the removed `train.py` script and `-v0 id`.
- `agents/rsl_rl_ppo_cfg.py`: the deprecated runner-level `empirical_normalization` was replaced by the two policy-level normalisation flags it used to set.
- Deleted `tricycle/soil.py`, an unused copy of `excavation/soil.py`; deleted the stray root file `__init__.py`.
- Added `.gitignore`; `__pycache__`/ and `luna_hifi_tasks.egg-info`/ are no longer tracked (they are regenerated by Python and pip).

Documentation

- `RUNBOOK.md`: a note on the body change under section 5.
- `docs/tricycle_manual/`: this document and its LaTeX source. Rebuild with `./build.sh` (needs TeX Live with `latexmk`, `tcolorbox`, `tikz`, `listings`).

Pulling these changes

If `git checkout` or `git pull` complains that local changes to `.pyc` or `egg-info` files would be overwritten, those are the formerly tracked generated files; discard them with `git checkout - .` (or `git stash`) and retry.

No reinstall is needed: `pyproject.toml` is unchanged by this revision, and the `egg-info` the pull removes is a build leftover rather than the metadata the CLI reads (Chapter 6). `tricycle.py` did change, so reconvert the asset:

```
.\isaacclab.bat -p -c "from luna_hifi_tasks.tricycle.tricycle import write_mjcf;  
    ↪ print(write_mjcf())"  
.\isaacclab.bat -p scripts\tools\convert_mjcf.py C:\Users\hanse\AppData\Local\Temp\  
    ↪ luna_hifi_assets\tricycle.xml C:\Users\hanse\Documents\Luna_HiFi\assets\  
    ↪ tricycle.usd
```

If the registry check ever comes back empty, reinstall once and it will be populated again:

```
.\isaacclab.bat -p -m pip install -e C:\Users\hanse\Documents\Luna_HiFi
```

Appendix C

Tensor shapes cheat sheet

N is the number of envs.

Expression	Shape	Meaning
<code>actions (to _pre_physics_step)</code>	$(N, 2)$	throttle, steer in $[-1, 1]$
<code>self._actions</code>	$(N, 2)$	clamped copy, also observed
<code>car.data.root_pos_w</code>	$(N, 3)$	root position, world frame
<code>car.data.root_lin_vel_b</code>	$(N, 3)$	root linear velocity, body frame
<code>car.data.root_ang_vel_b</code>	$(N, 3)$	root angular velocity, body frame
<code>car.data.projected_gravity_b</code>	$(N, 3)$	gravity direction in body frame
<code>car.data.joint_pos</code>	(N, J)	all joint positions; $J = 4$ here
<code>car.data.joint_vel</code>	(N, J)	all joint velocities
<code>car.data.default_root_state</code>	$(N, 13)$	pos(3) quat(4) lin vel(3) ang vel(3)
<code>car.data.default_joint_pos</code>	(N, J)	from the <code>cfg init_state</code>
<code>scene.env_origins</code>	$(N, 3)$	env origins, world frame
<code>obs["policy"]</code>	$(N, 15)$	the observation
<code>reward</code>	$(N,)$	float
<code>terminated, time_out</code>	$(N,)$	bool
<code>episode_length_buf</code>	$(N,)$	long, steps since reset
<code>soil.data.nodal_pos_w</code>	$(N, 720, 3)$	particle positions per env

The joint count J is 4: `steer`, `front_wheel`, `rear_left`, `rear_right`; the chassis free joint is the root state, not a joint in this table. Their order in the tensors comes from the USD and is what `find_joints` resolves.

Appendix D

Where to look in the Isaac Lab source

Paths are relative to the Isaac Lab checkout (`develop`).

Concept	File
CLI, entry-point loading	source/isaacrab/isaacrab/cli/__init__.py
train / play backends	source/isaacrab_rl/isaacrab_rl/entrypoints/ backends/train_rsl_rl.py, play_rsl_rl.py
common CLI args, -num_envs over-ride	source/isaacrab_rl/isaacrab_rl/entrypoints/common.py
Hydra presets, play_mode() call	source/isaacrab_tasks/isaacrab_tasks/utis/hydra.py
DirectRLEnv step/reset	source/isaacrab/isaacrab/envs/direct_rl_env.py
DirectRLEnvCfg fields, base	source/isaacrab/isaacrab/envs/direct_rl_env_cfg.py
play_mode	py
configclass	source/isaacrab/isaacrab/utis/configclass.py
spaces from ints	source/isaacrab/isaacrab/envs/utis/spaces.py
InteractiveScene	source/isaacrab/isaacrab/scene/interactive_scene.py
SimulationCfg	source/isaacrab/isaacrab/sim/simulation_cfg.py
articulation API	source/isaacrab/isaacrab/assets/articulation/ base_articulation.py
root/joint data fields	source/isaacrab/isaacrab/assets/rigid_object/ base_rigid_object_data.py
actuator cfgs	source/isaacrab/isaacrab/actuators/actuator_base_cfg.py, actuator_pd_cfg.py
NewtonCfg	source/isaacrab_newton/isaacrab_newton/physics/ newton_manager_cfg.py
MJWarpSolverCfg	source/isaacrab_newton/isaacrab_newton/physics/ mjwarp_manager_cfg.py
MPMSolverCfg	source/isaacrab_newton/isaacrab_newton/physics/ mpm_manager_cfg.py
MPM manager,	source/isaacrab_newton/isaacrab_newton/physics/ mpm_manager.py
reset_solver_state	source/isaacrab_newton/isaacrab_newton/physics/ newton_collision_cfg.py
collision pipeline cfg	source/isaacrab_newton/isaacrab_newton/assets/ mpm_object/mpm_object.py
MPMObject, reset	source/isaacrab_newton/isaacrab_newton/sim/ spawners/mpm/mpm_cfg.py, mpm.py
MPMGridCfg, material	source/isaacrab_newton/isaacrab_newton/sim/ schemas/schemas_cfg.py
Newton collision props	source/isaacrab_contrib/isaacrab_contrib/coupling/ coupler_cfg.py, coupler.py
coupler cfgs and manager	source/isaacrab_tasks/isaacrab_tasks/contrib/ ur10_particle_push/ur10_particle_push_env_cfg.py
reference coupled task	source/isaacrab_visualizers/isaacrab_visualizers/ newton/newton_visualizer_cfg.py
Newton GL visualiser cfg	source/isaacrab_rl/isaacrab_rl/rsl_rl/rl_cfg.py
rsl_rl cfg classes	source/isaacrab_rl/isaacrab_rl/rsl_rl/utis.py
rsl_rl cfg conversion	source/isaacrab_rl/isaacrab_rl/rsl_rl/vecenv_wrapper.py
rsl_rl wrapper	source/isaacrab/isaacrab/sim/converters/mjcf_converter.py, scripts/tools/convert_mjcf.py
MJCF converter	

In the Newton package (newton/_src/): the proxy coupler is solvers/coupled/solver_coupled_proxy.py, the implicit MPM solver is solvers/implicit_mpm/solver_implicit_mpm.py. In rsl_rl: runners/on_policy_runner.py and algorithms/ppo.py.

Appendix E

Numbers at a glance

The car (chassis frame, metres, kilograms)

Part	Centre (x, y, z)	Half-sizes / radius	Mass
left rail	$(0.035, 0.065, 0)$, yaw -19.4°	$(0.1961, 0.02, 0.015)$	0.8
right rail	$(0.035, -0.065, 0)$, yaw $+19.4^\circ$	$(0.1961, 0.02, 0.015)$	0.8
crossbar	$(-0.15, 0, 0)$	$(0.02, 0.15, 0.015)$	0.8
deck	$(-0.08, 0, 0)$	$(0.06, 0.065, 0.01)$	2.6
fork capsule	$(0.22, 0, -0.04)$	$r = 0.012$, half-length 0.02	0.1
front wheel	$(0.22, 0, -0.04)$	$r = 0.10$	0.5
rear left wheel	$(-0.15, 0.16, -0.04)$	$r = 0.10$	0.5
rear right wheel	$(-0.15, -0.16, -0.04)$	$r = 0.10$	0.5
chassis body	COM $(-0.054, 0, 0)$	inertia $(0.062, 0.044, 0.019)$	5.0
whole car		wheelbase 0.37, track 0.32	6.6

The soil

strip box (env-local)	$x \in [-0.4, 1.1]$, $y \in [-0.3, 0.3]$, $z \in [0.025, 0.085]$
voxel	0.05 m
particles per env	$30 \times 12 \times 2 = 720$, one per cell centre
particle mass	$0.05^3 \times 1800 = 0.225$ kg
material	$\rho = 1800$, friction 0.84, yield pressure 10^{12} Pa, cohesion 0
slab	$2.5 \times 1.6 \times 0.1$ m centred at $(0.35, 0, -0.05)$, kinematic, invisible
grid caps (active/leaf/lower/upper)	16384/8192/4096/1024, totals for all envs

Time

physics step	10 ms (rigid entry: 2 sub-steps of 5 ms; MPM: 1)
control step	20 ms (50 Hz)
episode	2 s = 100 control steps = 200 physics steps
rollout	50 control steps per env per iteration
training	300 iterations, checkpoint every 50